



深圳大学  
SHENZHEN UNIVERSITY



# 数字电路

江常坤

# 第一章 逻辑代数基础

## 1.1 概述

# 模拟量和数字量

- 在自然界中，存在着两类物理量：
  - 一类称为模拟量(Analog Quantity)，它具有时间上连续变化、值域内任意取值的特点，例如温度、压力、交流电压等就是典型的模拟量；
  - 另一类称为数字量(Digital Quantity)，它具有时间上离散变化（离散也就是不连续）、值域内只能取某些特定值的特点，例如训练场上运动员的人数、车间仓库里元器件的个数等就是典型的数字量。

# 知识要点

- 数字系统的基本概念；
- 常用计数制（二进制、八进制、十进制、十六进制）及其转换；
- 常用的几种代码（**BCD**码、**Gray**码和**ASCII**码）。

# 模拟信号和数字信号

- 在电子设备中，无论是数字量还是模拟量都是以电信号形式出现的。人们常常将表示模拟量的电信号叫作模拟信号(**Analog Signal**)，将表示数字量的电信号叫作数字信号(**Digital Signal**)。正弦波信号、话音信号就是典型的模拟信号，矩形波、方波信号就是典型的数字信号。

# 数 字 系 统

- 数字系统是一个能对数字信号进行加工、传递和存储的实体，它由实现各种功能的数字逻辑电路相互连接而成。

例如，数字计算机就是一种最具代表性的数字系统。

# 数字信号

- 若信号的变化在时间上和数值上都是离散的，或者说断续的，则称为**离散信号**。例如，学生成绩记录，工厂产品统计，电路开关的状态等。
- 离散信号的变化可以用不同的数字反映，所以又称为**数字信号**，简称为**数字量**。
- 数字系统中处理的是数字信号，当数字系统要与模拟信号发生联系时，必须经过**模/数(A/D)转换**和**数/模(D/A)转换**电路，对信号类型进行变换。

# 控制系统框图

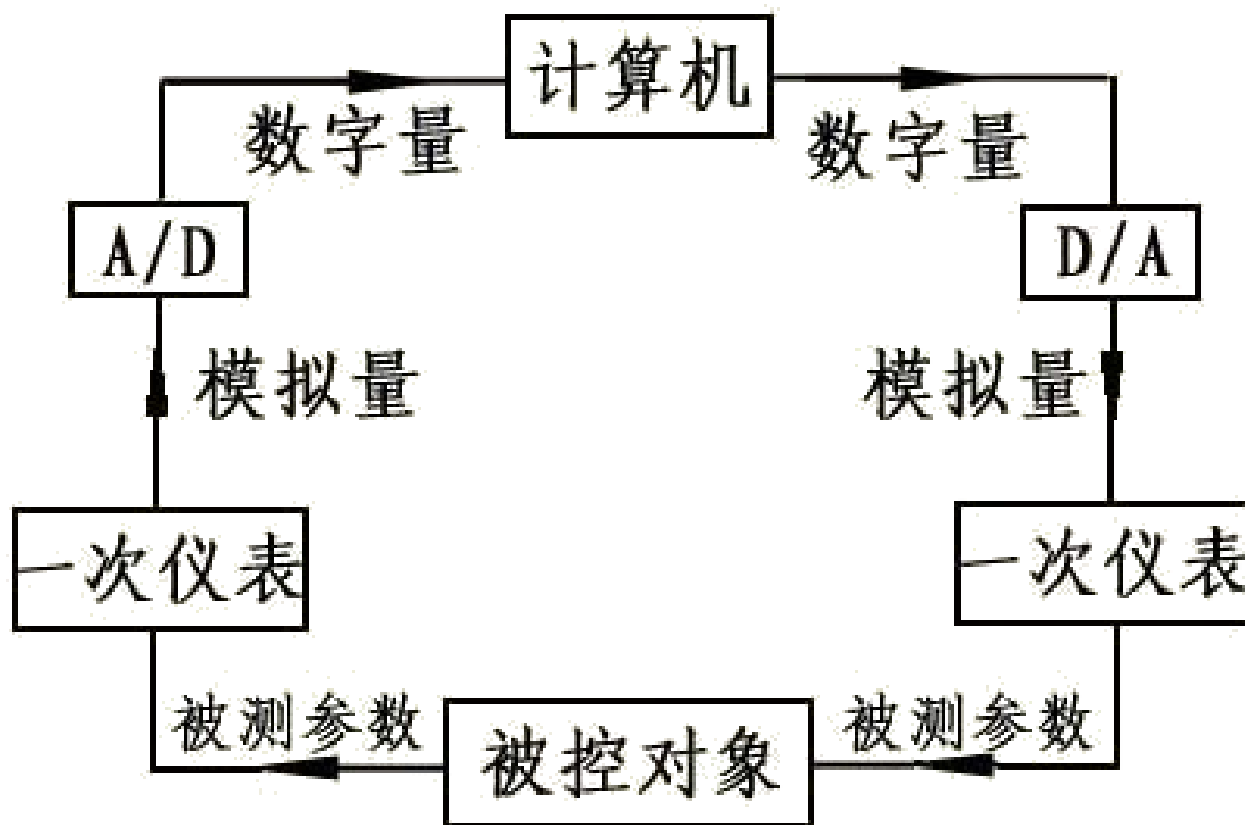


图 1.1



# 数字逻辑电路

用来处理数字信号的电子线路称为**数字电路**。

由于数字电路的各种功能是通过**逻辑运算和逻辑判断**来实现的，所以数字电路又称为**数字逻辑电路或者逻辑电路**。

# 数字逻辑电路特点

- 电路的基本工作信号是二值信号。它表现为电路中电压的“高”或“低”、开关的“接通”或“断开”、晶体管的“导通”或“截止”等两种稳定的物理状态。
- 电路中的半导体器件一般都工作在开、关状态，对电路进行研究时，主要关心输出和输入之间的逻辑关系。
- 电路结构简单、功耗低、便于集成制造和系列化生产。产品价格低廉、使用方便、通用性好。
- 由数字逻辑电路构成的数字系统工作速度快、精度高、功能强、可靠性好。随着半导体技术和工艺的发展，出现了数字集成电路，数字集成电路按照集成度的高低可分为小规模（SSI）、中规模（MSI）、大规模（LSI）和超大规模（VLSI）几种类型。

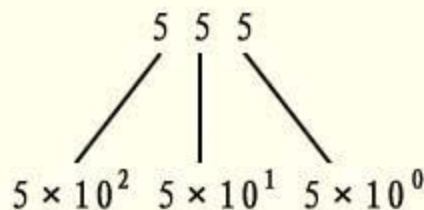
# 数制及其转换

- 人们与数据打交道的时候习惯于采用十进制，而计算机和其它数字系统中使用的是二进制。
- 为什么数字系统中要采用二进制？二进制有何优、缺点呢？如何克服它的缺点呢？人机联系时，如何进行不同进位制数之间的转换呢？

# 进位计数制

- 数制是人们对数量计数的一种统计规律。日常生活中广泛使用的是十进制，而数字系统中使用的是二进制。

十进制中采用了**0、1、...、9**共十个基本数字符号，进位规律是“逢十进一”。当用若干个数字符号并在一起表示一个数时，处在不同位置的数字符号，其值的含意不同。如：



即 同一个字符**5**从左到右所代表的值依次为**500、50、5**。

$$(555)_{10} = 5 \times 10^2 + 5 \times 10^1 + 5 \times 10^0$$

# 基数和位权

- 基数: 指计数制中所用到的数字符号的个数。在基数为 $R$ 的计数制中, 包含 $0$ 、 $1$ 、...、 $R-1$ 共 $R$ 个数字符号, 进位规律是“逢 $R$ 进一”, 称为 $R$ 进位计数制, 简称 $R$ 进制。
- 位权: 是指在某一种进位计数制表示的数中, 用来表明不同数位上数值大小的一个固定常数。不同数位有不同的位权, 某一个数位的数值等于这一位的数字符号乘上与该位对应的位权。 $R$ 进制数的位权是 $R$ 的整数次幂。例如, 十进制数的位权是 $10$ 的整数次幂, 其个位的位权是 $10^0$ , 十位的位权是 $10^1$ .....。

# R进制数表示法

- 一个R进制数N可以表示为：

$$\begin{aligned} N_R &= R^{n-1} \times r_{n-1} + R^{n-2} \times r_{n-2} + \cdots + R^1 \times r_1 + R^0 \times r_0 \\ &\quad + R^{-1} \times r_{-1} + R^{-2} \times r_{-2} + \cdots + R^{-m+1} \times r_{-m+1} + R^{-m} \times r_{-m} \\ &= \sum_{i=-m}^{n-1} R^i \times r_i \end{aligned}$$

其中，R表示基数；n为整数部分的位数；m为小数部分的位数； $r_i$ 为R进制中的一个数字符号，其取值范围为  $0 \leq r_i \leq R-1 (-m \leq i \leq n-1)$ 。

例如，十进制数 $(215.12)_{10}$ 的**多项式表示式**（也称**按权展开式**）为

$$(215.12)_{10} = 10^2 \times 2 + 10^1 \times 1 + 10^0 \times 5 + 10^{-1} \times 1 + 10^{-2} \times 2$$

# R进制的特点

- 有0、1、...、 $R-1$ 共 $R$ 个数字符号；
- “逢 $R$ 进一”，“10”表示 $R$ ；
- 位权是 $R$ 的整数次幂，第 $i$ 位的位权 $R^i$ 。

# 常用进制

- 十进制(Decimal System)
- 二进制(Binary System)
- 八进制(Octal System)
- 十六进制(Hexadecimal System)



# 常用进制

- 在计算机等数字设备中，用得最多的是二进制数和十六进制数，这是因为当前数字设备中所用的数字电路通常只有低电平和高电平两个状态，正好可用二进制数的 0 和 1 来表示。由于采用二进制来表示一个数时数位太多，所以常用与二进制数有简单对应关系的十六进制数(或八进制数)来表示一个数。
- 需要特别注意的是，在十六进制数中，用英文字母 A、B、C、D、E、F 分别表示十进制数的 10、11、12、13、14 和 15。

# 常用数制及其对应关系

| 项 目       | 十进制                 | 二进制              | 十六进制                            |
|-----------|---------------------|------------------|---------------------------------|
| 数符        | 0,1,2,3,4,5,6,7,8,9 | 0,1              | 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F |
| 第 $i$ 位的权 | $10^i$              | $2^i$            | $16^i$                          |
| 运算规则      | 逢 10 进 1, 借 1 为 10  | 逢 2 进 1, 借 1 为 2 | 逢 16 进 1, 借 1 为 16              |
| 对应关系      | 0                   | 0                | 0                               |
|           | 1                   | 1                | 1                               |
|           | 2                   | 10               | 2                               |
|           | 3                   | 11               | 3                               |
|           | 4                   | 100              | 4                               |
|           | 5                   | 101              | 5                               |
|           | 6                   | 110              | 6                               |
|           | 7                   | 111              | 7                               |
|           | 8                   | 1000             | 8                               |
|           | 9                   | 1001             | 9                               |
|           | 10                  | 1010             | A                               |
|           | 11                  | 1011             | B                               |
|           | 12                  | 1100             | C                               |
|           | 13                  | 1101             | D                               |
|           | 14                  | 1110             | E                               |
|           | 15                  | 1111             | F                               |
|           | 16                  | 10000            | 10                              |

# 数制转换

- 数制转换是指将一个数从一种进位制转换成另一种进位制。从实际应用出发，要求掌握二进制数与十进制数、八进制数、以及十六进制数之间的相互转换。

# 二进制数转换为十进制数

- 方法：多项式替代法

二进制数转换成十进制数时，只需将二进制数表示成按权展开式，并按十进制运算法则进行计算，所得结果即为该数对应的十进制数。

例如，

$$\begin{aligned}(10110.101)_2 &= 1 \times 2^4 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^{-1} + 1 \times 2^{-3} \\ &= 16 + 4 + 2 + 0.5 + 0.125 \\ &= (22.625)_{10}\end{aligned}$$

# 十进制数转换为二进制数

- **方法：基数乘除法**

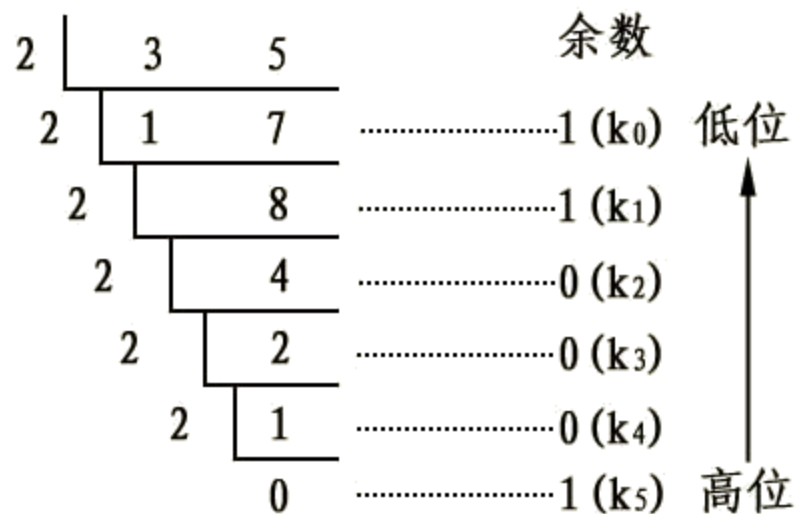
十进制数转换成二进制数时，应对整数和小数分别进行处理。整数转换采用“**除2取余**”的方法，小数转换采用“**乘2取整**”的方法。

# (1) 整数转换

- 将十进制整数**N**除以**2**，取余数计为**K<sub>0</sub>**；再将所得商除以**2**，取余数记为**K<sub>1</sub>**； ..... 。依此类推，直至商为**0**，取余数计为**K<sub>n-1</sub>**为止。即可得到与十进制整数**N**对应的**n**位二进制整数**K<sub>n-1</sub>...K<sub>1</sub>K<sub>0</sub>**。

# 举例

- 十进制整数**35**转换成二进制整数



即:  $(35)_{10} = (100011)_2$

## (2) 小数转换

- 将十进制小数**N**乘以**2**，取积的整数部分记为**K-1**；再将积的小数部分乘以**2**，取整数部分记为**K-2**；.....。依此类推，直至其小数部分为**0**或达到规定精度要求，取整数部分记作**K-m**为止。即可得到与**N**对应的**m**位二进制小数**0.K-1K-2...K-m**。



# 举例

- 将十进制小数0.6875转换成二进制小数。

|    |                               |                  |
|----|-------------------------------|------------------|
|    | 整数部分                          | 0. 6 8 7 5       |
|    |                               | × 2              |
| 高位 | 1 (K <sub>1</sub> ) . . . . . | <hr/> 1. 3 7 5 0 |
|    |                               | × 2              |
|    | 0 (K <sub>2</sub> ) . . . . . | <hr/> 0. 7 5 0 0 |
|    |                               | × 2              |
|    | 1 (K <sub>3</sub> ) . . . . . | <hr/> 1. 5 0 0 0 |
|    |                               | × 2              |
| 低位 | 1 (K <sub>4</sub> ) . . . . . | <hr/> 1. 0 0 0 0 |

即:  $(0.6875)_{10} = (0.1011)_2$

# 注意

- 当十进制小数不能用有限位二进制小数精确表示时。可根据精度要求，求出相应的二进制位数近似地表示。一般当要求二进制数取 $m$ 位小数时，可求出 $m+1$ 位，然后对最低位作“**0舍1入**”处理。

# 举例

- 例如，将十进制小数 0.323 转换成二进制小数 (保留4位小数)。

即:  $(0.323)_{10} = (0.0101)_2$

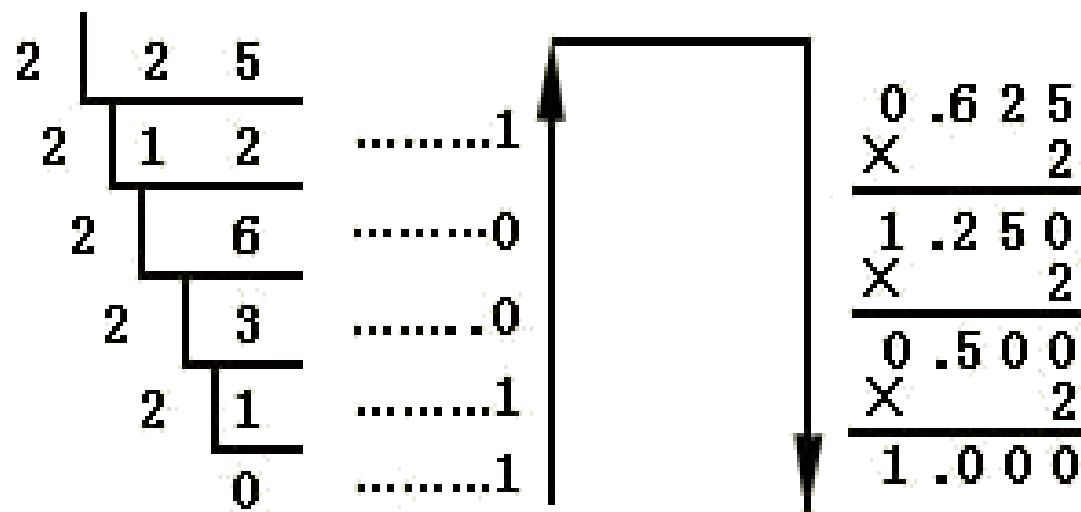
$$\begin{array}{r} 0.323 \\ \times 2 \\ \hline 0.646 \\ \times 2 \\ \hline 1.292 \\ \times 2 \\ \hline 0.584 \\ \times 2 \\ \hline 1.168 \\ \times 2 \\ \hline 0.336 \end{array}$$

# 注意

- 若一个十进制数既包含整数部分，又包含小数部分，则需将整数部分和小数部分分别转换，然后用小数点将两部分结果连到一起。

# 举例

- 例如，将十进制数 **25.625** 转换成二进制数。



即:  $(25.625)_{10} = (11001.101)_2$

# 二进制数与八进制数之间的转换

- 由于八进制中的基本数字符号0~7正好和3位二进制数的8种取值000~111对应。所以，二进制数与八进制数之间的转换可以按位进行。

二进制数转换成八进制数：以小数点为界，分别往高、往低每3位为一组，最后不足3位时用0补充，然后写出每组对应的八进制字符，即为相应八进制数。

# 举例

- 将二进制数11100101.01转换成八进制数。

即：  **$(11100101.01)_2 = (345.2)_8$**

# 八进制数转换成二进制数

- 只需将每位八进制数用**3**位二进制数表示。  
例如，将八进制数**56.7**转换成二进制数。
- 即： **$(56.7)_8 = (101\ 110 . 111)_2$**



# 二进制数与十六进制数之间的转换

- 二进制数与十六进制数之间的转换同样可以按位进行，只不过是4位二进制数对应1位十六进制数，即4位二进制数的取值 0000~1111 分别对应十六进制字符0~F。

二进制数转换成十六进制数：以小数点为界，分别往高、往低每4位为一组，最后不足4位时用0补充，然后写出每组对应的十六进制字符即可。

例如，将二进制数101110.011转换成十六进制数。

- 即： $(101110.011)_2 = (2E.6)_{16}$

# 十六进制数转换成二进制数

- 只需将每位十六进制数用**4**位二进制数表示。

例如，将十六进制数**5A.B**转换成二进制数。

- 即： **$(5A.B)_{16} = (1011010.1011)_2$**

# 其他转换

- 八-十转换、十六-十转换：多项式替代法
  - 只需将八或十六进制数表示成按权展开式，并按十进制运算法则进行计算，所得结果即为该数对应的十进制数。
- 十-八转换、十-十六转换：基数乘除法
  - 应对整数和小数分别进行处理。
  - 整数转换采用“**除8或16取余**”的方法，小数转换采用“**乘8或16取整**”的方法。

# 二进制算术运算

加法运算

$$\begin{array}{r} 1001 \\ + 0101 \\ \hline 1110 \end{array}$$

减法运算

$$\begin{array}{r} 1001 \\ - 0101 \\ \hline 0100 \end{array}$$

减法运算能通过补码运算转换为加法运算完成。

乘法运算

$$\begin{array}{r} 1001 \\ \times 0101 \\ \hline 1001 \\ 0000 \\ 1001 \\ 0000 \\ \hline 0101101 \end{array}$$

乘法运算可以通过若干次的“被乘数（或零）左移1位”和“被乘数（或零）与部分积相加”这两种操作完成；

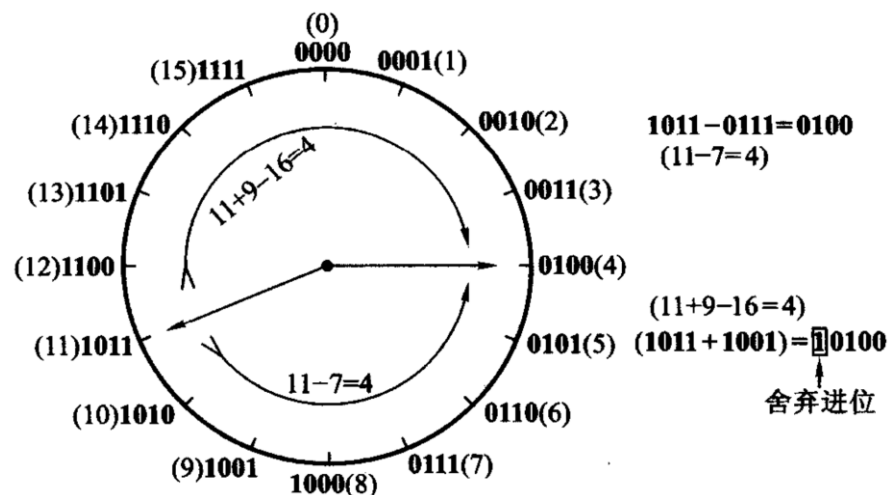
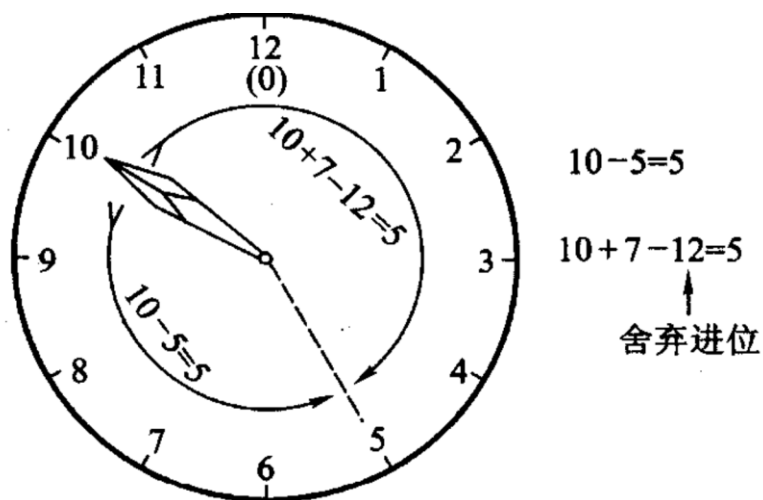
除法运算

$$\begin{array}{r} 1.11\cdots \\ 0101 \overline{) 1001} \\ \underline{0101} \\ 1000 \\ \underline{0101} \\ 0110 \\ \underline{0101} \\ 0010 \end{array}$$

除法运算能通过若干次的“除数右移1位”和“从被除数或余数中减去除数”这两种操作完成。

加、减、乘、除运算就全部可以用“移位”和“相加”两种操作实现了。利用上述特点能使运算电路的结构大为简化。这也是数字电路中普遍采用二进制算术运算的重要原因。

# 二进制算术运算



在舍弃进位的条件下，减去某个数可以用加上它的补码来代替。

负数补码： $(N)_{\text{补}} = (N)_{\text{反}} + 1$  求反操作电路易实现

# 几种常用的代码

- 在计算机和其它数字系统中，除了用二进制数表示十进制数之外，为了适应不同的需要、解决不同的问题，还引入了不同形式的编码。
- 用一组符号按一定规则表示给定字母、数字、符号等信息的方法称为编码，编码的结果称为代码。
- 三种常用的代码：
  - ◆ 二-十进制码（**BCD**码）；
  - ◆ **Gray**码；
  - ◆ **ASCII**码（字符代码）。

# BCD(Binary Coded Decimal)码

- 它是用二进制代码来表示十进制的10个数符，因此至少需要4位二进制数编码。当采用4位二进制编码时，共有16个码组，原则上可以从中任选10个来代表十进制的10个数符，多余的6个码组称为禁用码，平时不允许使用。

# 常用BCD码

| $N_{10}$ | 8421BCD 码 | 5421BCD 码 | 2421BCD 码 | 余 3 码 | 余 3 循环码 |
|----------|-----------|-----------|-----------|-------|---------|
| 0        | 0000      | 0000      | 0000      | 0011  | 0010    |
| 1        | 0001      | 0001      | 0001      | 0100  | 0110    |
| 2        | 0010      | 0010      | 0010      | 0101  | 0111    |
| 3        | 0011      | 0011      | 0011      | 0110  | 0101    |
| 4        | 0100      | 0100      | 0100      | 0111  | 0100    |
| 5        | 0101      | 1000      | 1011      | 1000  | 1100    |
| 6        | 0110      | 1001      | 1100      | 1001  | 1101    |
| 7        | 0111      | 1010      | 1101      | 1010  | 1111    |
| 8        | 1000      | 1011      | 1110      | 1011  | 1110    |
| 9        | 1001      | 1100      | 1111      | 1100  | 1010    |



# Gray码

- 格雷码是一种典型的无权循环码
- 循环码有两个特点，一个是相邻性，一个是循环性。相邻性是指任意两个相邻的代码中仅有1位取值不同，循环性是指首尾的两个代码也具有相邻性。

# 4位二进制格雷码

## 典型格雷码Gray Code

相邻编码，有且仅有一位代码不同。

可靠性编码

无权码

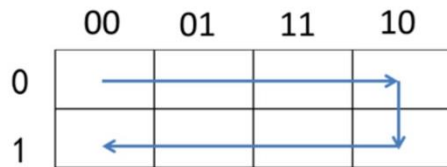
循环性

反射性

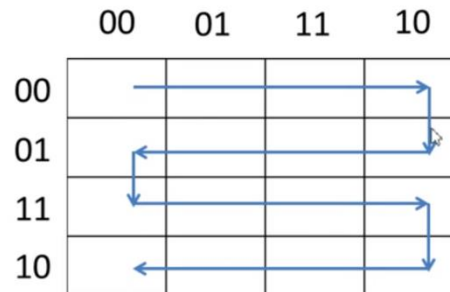
按 **Esc** 即可退出全屏模式



00、01、11、10



000、001、011、010、  
110、111、101、100



| 十进制 | 二进制  | 格雷码  |
|-----|------|------|
| 0   | 0000 | 0000 |
| 1   | 0001 | 0001 |
| 2   | 0010 | 0011 |
| 3   | 0011 | 0010 |
| 4   | 0100 | 0110 |
| 5   | 0101 | 0111 |
| 6   | 0110 | 0101 |
| 7   | 0111 | 0100 |
| 8   | 1000 | 1100 |
| 9   | 1001 | 1101 |
| 10  | 1010 | 1111 |
| 11  | 1011 | 1110 |
| 12  | 1100 | 1010 |
| 13  | 1101 | 1011 |
| 14  | 1110 | 1001 |
| 15  | 1111 | 1000 |

g.cdn.noip.cn\_38045888

# 二进制与格雷码转换

某二进制数为  $B_{n-1}B_{n-2} \cdots B_2B_1B_0$

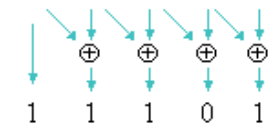
其对应的格雷码为  $G_{n-1}G_{n-2} \cdots G_2G_1G_0$

其中：最高位保留——  $G_{n-1} = B_{n-1}$

其他各位——  $G_i = B_{i+1} \oplus B_i \quad i=0, 1, 2, \dots, n-2$

例：二进制数为 1 0 1 1 0

格雷码为 1 1 1 0 1



异或运算：  
相同为0  
相异为1

某二进制格雷码为  $G_{n-1}G_{n-2} \cdots G_2G_1G_0$

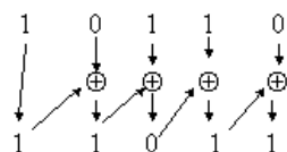
其对应的自然二进制码为  $B_{n-1}B_{n-2} \cdots B_2B_1B_0$

其中：最高位保留——  $B_{n-1} = G_{n-1}$

其他各位——  $B_{i-1} = G_{i-1} \oplus B_i \quad i=1, 2, \dots, n-1$

例：二进制格雷码为 1 0 1 1 0

自然二进制码为 1 1 0 1 1



异或运算：  
相同为0  
相异为1

# ASCII码

- **ASCII码是美国信息交换标准码，  
(American Standard Code for  
Information Interchange)。**

ASCII码用7位二进制码表示128种字符。

# ASCII码编码表

| $B_6 B_5 B_4$<br>$B_3 B_2 B_1 B_0$ | 000 | 001 | 010 | 011 | 100 | 101 | 110 | 111 |
|------------------------------------|-----|-----|-----|-----|-----|-----|-----|-----|
| 0000                               | NUL | DLE | SP  | 0   | @   | P   | ,   | p   |
| 0001                               | SOH | DC1 | !   | 1   | A   | Q   | a   | q   |
| 0010                               | STX | DC2 | "   | 2   | B   | R   | b   | r   |
| 0011                               | ETX | DC3 | #   | 3   | C   | S   | c   | s   |
| 0100                               | EOT | DC4 | \$  | 4   | D   | T   | d   | t   |
| 0101                               | ENQ | NAK | %   | 5   | E   | U   | e   | u   |
| 0110                               | ACK | SYN | &   | 6   | F   | V   | f   | v   |
| 0111                               | BEL | ETB | '   | 7   | G   | W   | g   | w   |
| 1000                               | BS  | CAN | (   | 8   | H   | X   | h   | x   |
| 1001                               | HT  | EM  | )   | 9   | I   | Y   | i   | y   |
| 1010                               | LF  | SUB | *   | :   | J   | Z   | j   | z   |
| 1011                               | VT  | ESC | +   | ;   | K   | [   | k   | {   |
| 1100                               | FF  | FS  | ,   | <   | L   | \   | l   |     |
| 1101                               | CR  | GS  | -   | =   | M   | ]   | m   | }   |
| 1110                               | SO  | RS  | .   | >   | N   | ↑   | n   | ~   |
| 1111                               | SI  | US  | /   | ?   | O   | ←   | o   | DEL |

控制符的含义如下：

NUL Null空白

SOH Start of Heading标题开始

STX Start of Text正文开始

ETX End of Text正文结束

EOT End of Transmission传输结束

ENQ Enquiry询问

ACK Acknowledge确认

BEL Bell响铃(告警)

BS Backspace退一格

DC1 Device Control 1设备控制1

DC2 Device Control 2设备控制2

DC3 Device Control 3设备控制3

DC4 Device Control 4设备控制4

NAK Negative Acknowledge否认

SYN Synchronous Idle同步空传

ETB End of Transmission Block块结束

CAN Cancel取消

EM End of Medium纸尽

|     |                           |     |                       |
|-----|---------------------------|-----|-----------------------|
| HT  | Horizontal Tabulation水平列表 | SUB | Substitute替换          |
| LF  | Line Feed换行               | ESC | Escape脱离              |
| VT  | Vertical Tabulation垂直列表   | FS  | File Separator文件分离符   |
| FF  | Form Feed 走纸              | GS  | Group Separator字组分离符  |
| CR  | Carriage Return回车         | RS  | Record Separator记录分离符 |
| SO  | Shift Out移出               | US  | Unit Separator单元分离符   |
| SI  | Shift In移入                | SP  | Space空格               |
| DLE | Data Link Escape数据链路换码    | DEL | Delete删除              |

# 作业:

P32: 1-1(3);1-2(3);1-4(3).

表 1-22 为函数 F 的真值表。

$$F = A + \overline{B}\overline{C}$$

## 习 题

1-1 将下列十进制数转换为二进制数、八进制数和十六进制数。

- (1)  $22_{10}$       (2)  $108_{10}$       (3)  $13.125_{10}$       (4)  $131.625_{10}$

1-2 将下列二进制数转换为十进制数、八进制数和十六进制数。

- (1)  $101101_2$       (2)  $11100101_2$       (3)  $101.0011_2$       (4)  $100111.101_2$

1-3 将下列八进制数转换为十进制数、二进制数和十六进制数。

- (1)  $16_8$       (2)  $172_8$       (3)  $61.53_8$       (4)  $126.74_8$

1-4 将下列十六进制数转换为十进制数、二进制数和八进制数。

- (1)  $2A_{16}$       (2)  $B2F_{16}$       (3)  $D3.E_{16}$       (4)  $1C3.F9_{16}$

用真值表证明下列逻辑等式。





深圳大学  
SHENZHEN UNIVERSITY



# 数字电路

江常坤

## 1.2 逻辑代数的基本运算和门电路

# 知识要点

- 掌握逻辑代数的基本概念（包括变量、运算等）；
- 掌握逻辑代数的基本运算和门电路。

# 逻辑代数

- 逻辑代数又称布尔代数
- 逻辑代数研究逻辑变量间的相互关系

# 逻辑变量

- 逻辑代数和普通代数一样，也是用字母表示其值可以变化的量，即变量。需注意的是：
  1. 在普通代数中，变量的取值可以是任意实数，而逻辑代数是一种二值代数系统，即任何逻辑变量的取值只有两种可能性--取值0或取值1。
  2. 逻辑值0和1不再像普通代数中那样具有数量的概念，而是用来表征矛盾的双方和判断事件真伪的形式符号，无大小、正负之分。在数字系统中，开关的接通与断开，电压的高和低，信号的有和无，晶体管的导通与截止等两种稳定的物理状态，均可用1和0这两种不同的逻辑值来表征。

# 基本逻辑运算

- 逻辑代数有与(AND)、或(OR)、非(NOT)三种基本运算(也分别称为逻辑乘、逻辑加和逻辑非), 其运算符分别为“.”、“+”和“-”。与运算符“.”通常可以省略。
- 逻辑运算的功能常用真值表(Truth Table)来描述。将自变量的各种可能取值及其对应的函数值F列在一张表上, 就构成了真值表。

# “与” 运算(AND)

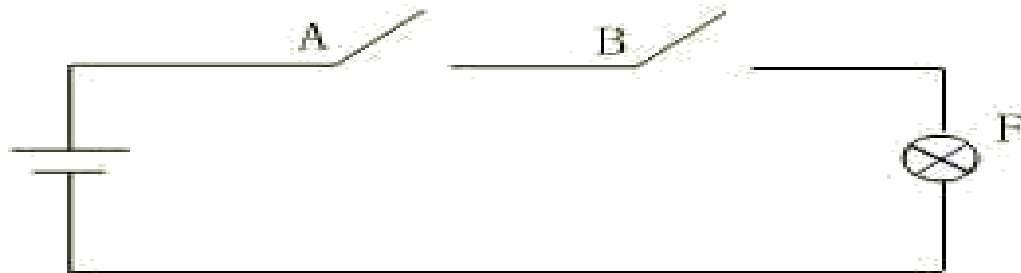
- 在逻辑问题中，如果决定某一事件发生的多个条件必须同时具备，事件才能发生，则这种因果关系称之为“与”逻辑。在逻辑代数中，“与”逻辑关系用“与”运算描述。“与”运算又称为逻辑乘(Logic Multiplication)，其运算符号为“·”。两变量“与”运算关系可表示为

$$F = A \cdot B$$

读作“F等于A与B”。意思是：若A、B均为1，则F为1；否则，F为0。

# 与的逻辑电路

- 两个开关**串联**控制同一个灯。显然，仅当两个开关均闭合时，灯才能亮，否则，灯灭。假定开关闭合状态用1表示，开关断开状态用0表示，灯亮用1表示，灯灭用0表示，则电路中灯F和开关A、B之间的关系即为“与”运算关系。





# 与的逻辑真值表

| A | B | $F=A \cdot B$ |
|---|---|---------------|
| 0 | 0 | 0             |
| 0 | 1 | 0             |
| 1 | 0 | 0             |
| 1 | 1 | 1             |

# “与”运算的运算法则

$$0 \cdot 0 = 0 \quad 1 \cdot 0 = 0$$

$$0 \cdot 1 = 0 \quad 1 \cdot 1 = 1$$

# “或”运算（OR）

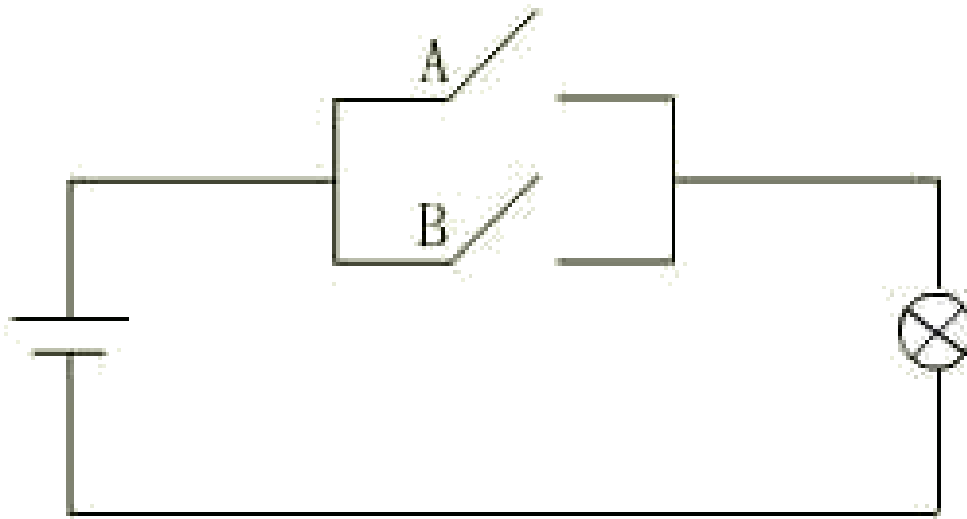
- 在逻辑问题的描述中，如果决定某一事件是否发生的多个条件中，只要有一个或一个以上条件成立，事件便可发生，则这种因果关系称之为“或”逻辑。逻辑代数中，“或”逻辑用“或”运算描述。“或”运算又称逻辑加(Logic addition)，其运算符号为“+”。两变量“或”运算的关系表示为

$$F = A + B$$

读作“F等于A或B”。意思是：**A、B**中只要有一个为**1**，则**F**为**1**；仅当**A、B**均为**0**时，**F**才为**0**。

# 或的逻辑电路

- 开关**A**和**B**并联控制灯**F**。可以看出，当开关**A**、**B**中有一个闭合或者两个均闭合时，灯**F**即亮。因此，灯**F**与开关**A**、**B**之间的关系是“或”逻辑关系



# 或的逻辑真值表

| A | B | $F=A+B$ |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 1       |
| 1 | 0 | 1       |
| 1 | 1 | 1       |

# “或”运算的运算法则

$$0 + 0 = 0 \quad 1 + 0 = 1$$

$$0 + 1 = 1 \quad 1 + 1 = 1$$

# “非”运算（NOT）

- 在逻辑问题中，如果某一事件的发生取决于条件的否定，即事件与事件发生的条件之间构成矛盾，则这种因果关系称为“非”逻辑。在逻辑代数中，“非”逻辑用“非”运算描述。“非”运算也叫求反运算或者逻辑否定(Logic Negation)。其运算符号为“-”。“非”运算的逻辑关系可表示为

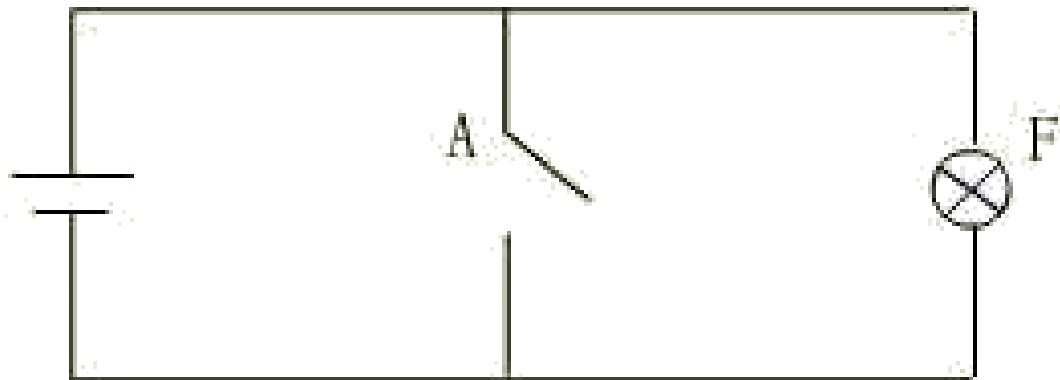
—

$$F=A$$

- 读作“F等于A非”。意思是：若A为0，则F为1；若A为1，则F为0。

# 非的逻辑电路

- **开关与灯并联**。显然，仅当开关断开时，灯亮；一旦开关闭合，则灯灭。令开关断开用0表示，开关闭合用1表示，灯亮用1表示，灯灭用0表示，则电路中灯F与开关A的关系所示“非”运算关系。





# 非的逻辑真值表

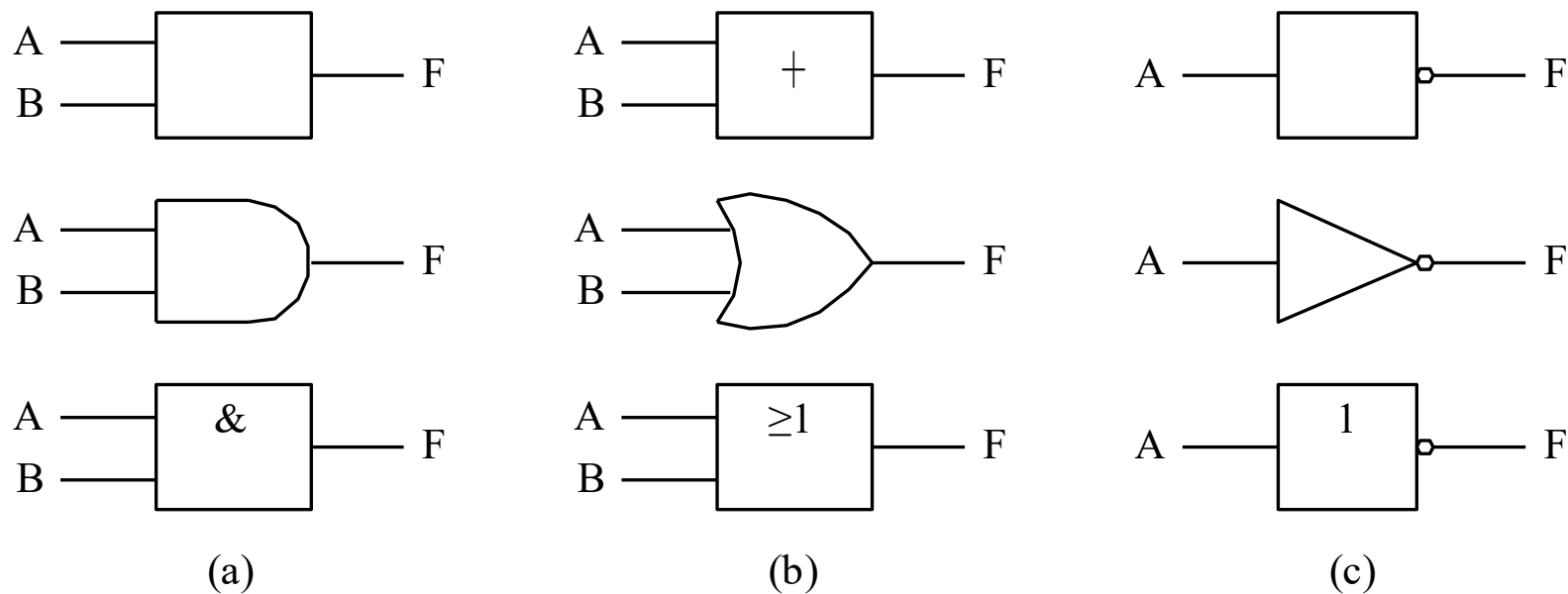
| A | $F=A$ |
|---|-------|
| 0 | 1     |
| 1 | 0     |

# “非”运算的运算法则

$$\overline{0}=1$$

$$\overline{1}=0$$

# 常用门电路逻辑符号



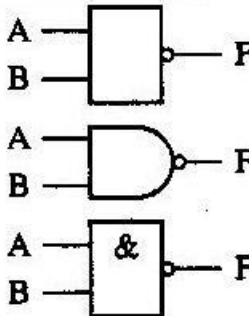
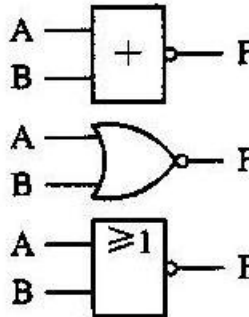
与、或、非门的逻辑符号

(a) 与门符号； (b) 或门符号； (c) 非门符号

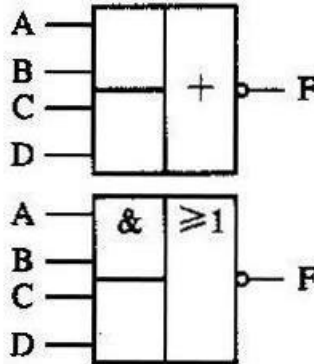
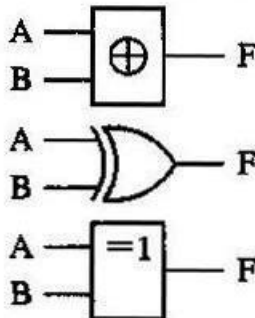
# 其他常见逻辑运算

- 将与、或、非三种基本的逻辑运算进行组合，可以得到各种形式的复合逻辑运算。
- 最常用的几种复合逻辑运算
  - “与非(NAND)”运算
  - “或非(NOR)”运算
  - “异或(XOR)”运算
  - “同或(XNOR)”运算
  - “与或非(AND-OR-OT)”运算

# 复合逻辑运算与常用逻辑门

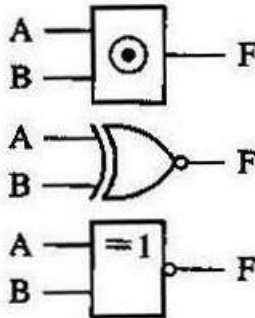
| 名称   | 代数式                        | 真值表                                                                                                                                                                                                                | 逻辑门符号 | 基本特征 |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
|------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|------|---|---|---|---|---|---|---|---|---|---|---|---|---|--------------------------------------------------------------------------------------|---------------------------|
| 与非运算 | $F = \overline{A \cdot B}$ | <table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table> | A     | B    | F | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |   | 全部输入都为 1 时，<br>输出 $F = 0$ |
| A    | B                          | F                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 0    | 0                          | 1                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 0    | 1                          | 1                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 1    | 0                          | 1                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 1    | 1                          | 0                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 或非运算 | $F = \overline{A + B}$     | <table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table> | A     | B    | F | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 |  | 全部输入都为 0 时，<br>输出 $F = 1$ |
| A    | B                          | F                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 0    | 0                          | 1                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 0    | 1                          | 0                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 1    | 0                          | 0                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |
| 1    | 1                          | 0                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                           |

# 复合逻辑运算与常用逻辑门

| 与或非运算 | $F = \overline{AB+CD}$                                | <table> <tr> <th>AB</th> <th>CD</th> <th>F</th> </tr> <tr> <td>0</td> <td>0</td> <td>1</td> </tr> <tr> <td>0</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>0</td> </tr> <tr> <td>1</td> <td>1</td> <td>0</td> </tr> </table> | AB | CD | F | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0 |    | 全部与项都为 0 时，<br>输出 $F = 1$                                                                                                        |
|-------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|---|---|---|---|---|---|---|---|---|---|---|---|---|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| AB    | CD                                                    | F                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 0     | 0                                                     | 1                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 0     | 1                                                     | 0                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 1     | 0                                                     | 0                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 1     | 1                                                     | 0                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 异或运算  | $F = A \oplus B$<br>$= \overline{A}B + A\overline{B}$ | <table> <tr> <th>A</th> <th>B</th> <th>F</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>0</td> </tr> </table>   | A  | B  | F | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |  | 两个输入变量取值不同时， $F=1$<br>特别有：<br>$A \oplus 0 = A$<br>$A \oplus 1 = \overline{A}$<br>$A \oplus A = 0$<br>$A \oplus \overline{A} = 1$ |
| A     | B                                                     | F                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 0     | 0                                                     | 0                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 0     | 1                                                     | 1                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 1     | 0                                                     | 1                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |
| 1     | 1                                                     | 0                                                                                                                                                                                                                                              |    |    |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                      |                                                                                                                                  |

# 复合逻辑运算与常用逻辑门

续表

| 名称            | 代数式                                                                          | 真值表                                                                                                                                                                                                                | 逻辑门符号 | 基本特征 |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                     |                                                                                                                   |
|---------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|------|---|---|---|---|---|---|---|---|---|---|---|---|---|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 同或运算<br>(异或非) | $F = A \odot B$ $= \overline{A \oplus B}$ $= \overline{A} \overline{B} + AB$ | <table><tr><td>A</td><td>B</td><td>F</td></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table> | A     | B    | F | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 1 |  | 两个输入变量取值相同时, $F=1$<br>特别有:<br>$A \odot 0 = \overline{A}$<br>$A \odot 1 = A$<br>$A \odot A = 1$<br>$A \odot A = 0$ |
| A             | B                                                                            | F                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                     |                                                                                                                   |
| 0             | 0                                                                            | 1                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                     |                                                                                                                   |
| 0             | 1                                                                            | 0                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                     |                                                                                                                   |
| 1             | 0                                                                            | 0                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                     |                                                                                                                   |
| 1             | 1                                                                            | 1                                                                                                                                                                                                                  |       |      |   |   |   |   |   |   |   |   |   |   |   |   |   |                                                                                     |                                                                                                                   |

## 1.3 逻辑代数的公式和规则



# 基本公式

- $0 \cdot 0 = 0$

$$0 + 0 = 0$$

- $0 \cdot 1 = 0$

$$0 + 1 = 1$$

- $1 \cdot 1 = 1$

$$1 + 1 = 1$$

- $\overline{0} = 1$

$$\overline{1} = 0$$

# 基本公式

| 名称    | 公式 1                                                  | 公式 2                                                          |
|-------|-------------------------------------------------------|---------------------------------------------------------------|
| 交换律   | $A+B = B+A$                                           | $AB = BA$                                                     |
| 结合律   | $A+(B+C) = (A+B)+C$                                   | $A(BC) = (AB)C$                                               |
| 分配律   | $A+BC = (A+B)(A+C)$                                   | $A(B+C) = AB+AC$                                              |
| 互补律   | $A+\bar{A} = 1$                                       | $A\bar{A} = 0$                                                |
| 0-1 律 | $A+0 = A$<br>$A+1 = 1$                                | $A \cdot 1 = A$<br>$A \cdot 0 = 0$                            |
| 对合律   | $\bar{\bar{A}} = A$                                   | $\bar{\bar{A}} = A$                                           |
| 重叠律   | $A+A = A$                                             | $AA = A$                                                      |
| 吸收律   | $A+AB = A$<br>$A+\bar{A}B = A+B$<br>$AB+A\bar{B} = A$ | $A(A+B) = A$<br>$A(\bar{A}+B) = AB$<br>$(A+B)(A+\bar{B}) = A$ |
| 包含律   | $AB+\bar{A}C+BC = AB+\bar{A}C$                        | $(A+B)(\bar{A}+C)(B+C) = (A+B)(\bar{A}+C)$                    |
| 反演律   | $\overline{A+B} = \bar{A}\bar{B}$                     | $\overline{AB} = \bar{A}+\bar{B}$                             |

例：证明分配律、吸收律和包含律的正确性。

证明：

$$(A + B)(A + C) = A + BC$$

$$A + AB = A$$

$$A + \overline{A}B = A + B$$

$$AB + A\overline{B} = A$$

$$AB + \overline{A}C + BC = AB + \overline{A}C$$

例：证明分配律、吸收律和包含律的正确性。

证明：

$$\begin{aligned}(A+B)(A+C) &= AA + AC + BA + BC = A + AC + AB + BC \\ &= A(1+C+B) + BC = A \cdot 1 + BC = A + BC\end{aligned}$$

$$A + AB = A(1+B) = A \cdot 1 = A$$

$$\begin{aligned}A + \bar{A}B &= A(B + \bar{B}) + \bar{A}B = AB + A\bar{B} + \bar{A}B = AB + AB + A\bar{B} + \bar{A}B \\ &= A(B + \bar{B}) + B(A + \bar{A}) = A + B\end{aligned}$$

$$AB + A\bar{B} = A(B + \bar{B}) = A$$

$$\begin{aligned}AB + \bar{A}C + BC &= AB + \bar{A}C + (A + \bar{A})BC = AB + \bar{A}C + ABC + \bar{A}BC \\ &= AB(1+C) + \bar{A}C(1+B) = AB + \bar{A}C\end{aligned}$$



深圳大学  
SHENZHEN UNIVERSITY



# 数字电路

江常坤

# 常用公式

- $A + A \cdot B = A$  (吸收律)
- $A + \bar{A} \cdot B = A + B$  (吸收律)
- $A \cdot B + \bar{A} \cdot C = A \cdot B + \bar{A} \cdot C + B \cdot C$  (包含律)
- $A \cdot B + \bar{A} \cdot C = A \cdot B + \bar{A} \cdot C + B \cdot C \cdot D$  (包含律)

# 运算规则

## 1) 代入规则

- 对于任何一个逻辑等式，以某个逻辑变量或逻辑函数同时取代等式两端的任何一个逻辑变量A后，等式依然成立。这就是代入规则。
- 利用代入规则，可以方便地扩展公式。例如，可以把摩根定律扩展到含有n个变量的等式：

$$\overline{A_1 + A_2 + \cdots + A_n} = \overline{A_1} \cdot \overline{A_2} \cdots \overline{A_n}$$
$$\overline{A_1 \cdot A_2 \cdots A_n} = \overline{A_1} + \overline{A_2} + \cdots + \overline{A_n}$$

下面以三变量为例进行证明：

$$\overline{A + B + C} = \overline{A} \cdot \overline{B + C} = \overline{A} \cdot \overline{B} \cdot \overline{C}$$

$$\overline{A \cdot B \cdot C} = \overline{A} + \overline{B \cdot C} = \overline{A} + \overline{B} + \overline{C}$$



## 2) 对偶规则

设 $F$ 为一个逻辑函数表达式，若将 $F$ 中的“与”、“或”运算符互换(即 $\cdot$ 变为 $+$ ， $+$ 变为 $\cdot$ )，常量0、1互换(即0变为1，1变为0)，所得到的新表达式就叫做函数 $F$ 的对偶式(Duality Expression)或对偶函数(Duality Function)，常用 $F_d$ 表示。

如果两个逻辑函数表达式相等，那么它们的对偶式也一定相等。这就是对偶规则。

利用对偶规则，不仅可以帮助人们证明逻辑等式，而且可以帮助人们减少公式的记忆量。例如，基本公式表中的公式1和公式2就互为对偶式，只需记忆其中一边的公式就可以了。

# 基本公式

| 名称    | 公式 1                                                  | 公式 2                                                          |
|-------|-------------------------------------------------------|---------------------------------------------------------------|
| 交换律   | $A+B = B+A$                                           | $AB = BA$                                                     |
| 结合律   | $A+(B+C) = (A+B)+C$                                   | $A(BC) = (AB)C$                                               |
| 分配律   | $A+BC = (A+B)(A+C)$                                   | $A(B+C) = AB+AC$                                              |
| 互补律   | $A+\bar{A} = 1$                                       | $A\bar{A} = 0$                                                |
| 0-1 律 | $A+0 = A$<br>$A+1 = 1$                                | $A \cdot 1 = A$<br>$A \cdot 0 = 0$                            |
| 对合律   | $\bar{\bar{A}} = A$                                   | $\bar{\bar{A}} = A$                                           |
| 重叠律   | $A+A = A$                                             | $AA = A$                                                      |
| 吸收律   | $A+AB = A$<br>$A+\bar{A}B = A+B$<br>$AB+\bar{A}B = B$ | $A(A+B) = A$<br>$A(\bar{A}+B) = AB$<br>$(A+B)(A+\bar{B}) = A$ |
| 包含律   | $AB+\bar{A}C+BC = AB+\bar{A}C$                        | $(A+B)(\bar{A}+C)(B+C) = (A+B)(\bar{A}+C)$                    |
| 反演律   | $\overline{A+B} = \bar{A}\bar{B}$                     | $\overline{AB} = \bar{A}+\bar{B}$                             |

例：求  $F = A \cdot \overline{B} \cdot C + B \cdot \overline{C} + \overline{\overline{D}}$  的对偶函数。

例：求  $F = A \cdot \overline{B} \cdot C + B \cdot \overline{C} + \overline{D}$  的对偶函数。

解  $F_d = A + (\overline{B} + C) \cdot (B + C \cdot \overline{D})$

求对偶函数时，**要注意保持原式中的运算次序不变**。本例 $F_d$ 表达式中的括号就是为了保证原式中的运算次序不变。原来大非号下面为两项相或，因此变号后应为两项相与。

### 3) 反演规则

在逻辑代数中，常将逻辑函数 $F$ 叫作原函数，将 $\overline{F}$ 叫作 $F$ 的反函数或补函数，将由原函数求反函数的过程叫作“反演(Reversal Development)”或“求反”。若 $A$ 是函数 $F$ 的一个自变量，则称 $\overline{A}$ 为原变量 $A$ 的反变量。

将一个逻辑函数表达式 $F$ 中的“与”、“或”运算符互换，常量0、1互，原变量与反变量互换，就可得到 $F$ 的反函数 $\overline{F}$ 。这就是反演规则。

利用反演规则求反函数 $\overline{F}$ 时，不仅要注意运算的优先顺序，而且还要注意只有单个变量的反变量才变为原变量，而对于多个变量组合后的“非”号不能变反。

例：写出上例中F的反函数 $\overline{F}$ 的表达式并用反演律验证其正确性。

$$F = A \cdot \overline{B} \cdot C + B \cdot \overline{C} + \overline{\overline{D}}$$

例: 写出上例中F的反函数 $\bar{F}$ 的表达式并用反演律验证其正确性。

$$F = A \cdot \bar{B} \cdot C + B \cdot \bar{C} + \bar{D}$$

解 利用反演规则，有

$$\bar{F} = \bar{A} + \overline{(B + \bar{C}) \cdot (\bar{B} + \bar{\bar{C}} \cdot D)}$$

利用德摩根定律（反演律），有

$$\begin{aligned}\bar{F} &= \overline{A \cdot \bar{B} \cdot C + B \cdot \bar{C} + \bar{D}} = \bar{A} + \bar{B} \cdot C + \overline{B \cdot \bar{C} + \bar{D}} = \bar{A} + \bar{B} \cdot C + \overline{B \cdot \bar{C}} + \bar{\bar{D}} \\&= \bar{A} + \overline{(B + \bar{C})} + B \cdot (\bar{\bar{C}} \cdot D) = \bar{A} + \overline{(B + \bar{C})} + \overline{(\bar{B} + \bar{\bar{C}} \cdot D)} \\&= \bar{A} + \overline{(B + \bar{C}) \cdot (\bar{B} + \bar{\bar{C}} \cdot D)}\end{aligned}$$

二者是完全相等的。因此，利用反演规则所得到的反函数是正确的。

作业:

P33: 1-6(4);  
1-7(7);  
1-8(4).



## 1.4 逻辑函数常用的描述方法及相互间的转换

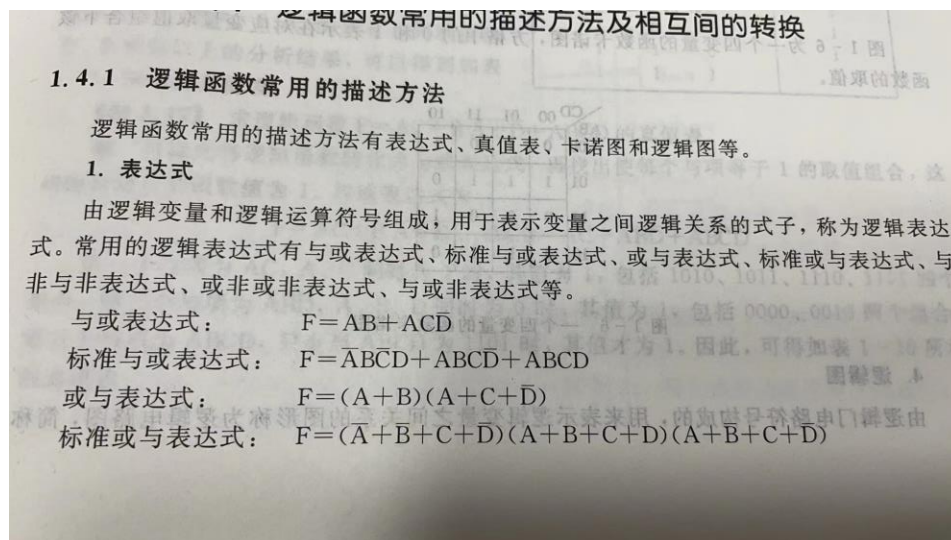
# 逻辑函数常用的描述方法

- 表达式
- 真值表
- 卡诺图
- 逻辑图
- 波形图

# 表达式

- 由逻辑变量和逻辑运算符号组成，用于表示变量之间逻辑关系的式子，称为逻辑表达式。

- 与或表达式
- 标准与或表达式
- 或与表达式
- 标准或与表达式
- 与非与非表达式
- 或非或非表达式
- 与或非表达式



# 表达式

- 由逻辑变量和逻辑运算符号组成，用于表示变量之间逻辑关系的式子，称为逻辑表达式。
  - 与或表达式
  - 标准与或表达式
  - 或与表达式
  - 标准或与表达式
  - 与非与非表达式
  - 或非或非表达式
  - 与或非表达式

14

数字电路与逻辑设计

与非与非表达式:  $F = \overline{\overline{A} \overline{B} \overline{C} \overline{D}}$

或非或非表达式:  $F = \overline{\overline{A} + \overline{B} + \overline{C} + \overline{D}}$

与或非表达式:  $F = \overline{AB + CD}$

## 2. 真值表

用来反映变量所有取值组合及对应函数值的表格，称为真值表。

例如，在一个判奇电路中，当 A、B、C 三个变量中有奇数个 1 时，输出 F 为 1；否则，输出 F 为 0。据

# 逻辑函数的标准形式

## 1) 标准积之和式 （标准与或表达式）

在逻辑代数中，全部输入变量均以原变量或反变量的形式出现1次且仅出现1次的“乘积项(与项)”称为“标准积项”。 $n$ 个变量的逻辑函数最多可以有 $2^n$ 个标准积项。以两个变量A、B的函数为例，它最多可以有4个标准积项： $\overline{A}\overline{B}$ 、 $\overline{A}B$ 、 $A\overline{B}$  和 $AB$ 。诸如  $\overline{A}$ 、 $A\overline{A}$ 、 $A\overline{B}B$  等形式的乘积项都不是标准积项。

标准积项有时也称为“**最小项(Minterm)**”，并用小写英文字母m加序号下标*i*的形式来简记。**序号*i***的确定方法是，将最小项中的变量按序排列后，**原变量用1表示，反变量用0表示**，得到1组二进制数，将其转换为等值的**十进制数**，就是序号*i*。例如，**二变量函数F(A,B)**的4个最小项为

$$m_0 = \overline{A}\overline{B}(i = (00)_2 = 0)$$

$$m_1 = \overline{A}B(i = (01)_2 = 1)$$

$$m_2 = A\overline{B}(i = (10)_2 = 2)$$

$$m_3 = AB(i = (11)_2 = 3)$$

全部由标准积项逻辑加构成的“**积之和(SOP)**”逻辑表达式称为“**标准积之和式(Canonical SOP Expression)**”或“**标准与或式**”。由于标准积项也称为最小项，所以有时也把标准积之和式称为“**最小项表达式**”。例如，下面就是一个三变量逻辑函数F(A,B,C)的“标准积之和式”的三种表示形式：

$$\begin{aligned} F(A, B, C) &= \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}C + ABC\overline{C} && \text{(变量型)} \\ &= m_0 + m_2 + m_5 + m_6 && \text{(m型)} \\ &= \sum m(0, 2, 5, 6) && \text{(\Sigma m型)} \end{aligned}$$

## 2) 标准和之积式 （标准或与表达式）

在逻辑代数中，全部输入变量均以原变量或反变量的形式出现1次且仅出现1次的“和项(或项)”称为“标准和项”。 $n$ 个变量的逻辑函数最多可以有 $2^n$ 个标准和项。以两个变量 $A$ 、 $B$ 的函数为例，它最多可以有4个标准和项： $\overline{A} + \overline{B}$ 、 $\overline{A} + B$ 、 $A + B$ 和 $A + \overline{B}$ 。诸如 $A$ 、 $A + \overline{A}$ 、 $A + B + \overline{B}$ 等形式的和项都不是标准和项。



标准和项有时也称为“**最大项**(Maxterm)”，并用大写英文字母**M**加序号下标*i*的形式来简记。序号*i*的确定方法是，将最大项中的变量按序排列后，**原变量用0表示，反变量用1表示**，得到1组二进制数，将其转换为等值的**十进制数**，就是序号*i*。例如，二变量函数F(A,B)的4个最大项为

$$M_0 = A + B (i = (00)_2 = 0)$$

$$M_1 = A + \bar{B} (i = (01)_2 = 1)$$

$$M_2 = \bar{A} + B (i = (10)_2 = 2)$$

$$M_3 = \bar{A} + \bar{B} (i = (11)_2 = 3)$$

全部由标准和项逻辑乘后构成的“**和之积(POS)**”逻辑表达式称为“**标准和之积式(Canonical POS Expression)**”或“**标准或与式**”。由于标准和项也称为最大项，所以有时也把标准和之积式称为“**最大项表达式**”。例如，下面就是一个三变量逻辑函数 $F(A,B,C)$ 的“标准和之积式”的三种表示形式：

$$\begin{aligned} F(A, B, C) &= (A + B + \overline{C})(A + \overline{B} + \overline{C})(\overline{A} + B + C)(\overline{A} + \overline{B} + \overline{C}) && \text{(变量型)} \\ &= M_1 M_3 M_4 M_7 && \text{(M型)} \\ &= \prod M(1, 3, 4, 7) && \text{(\prod M型)} \end{aligned}$$

### 3) 最小项与最大项的性质

(1) 全部最小项之和恒为1，全部最大项之积恒为0，即

$$\sum_{i=0}^{2^n-1} m_i = 1$$

$$\prod_{i=0}^{2^n-1} M_i = 0$$

$$F(A, B, C) = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}C + ABC$$

$$F(A, B, C) = (A + B + \overline{C})(A + \overline{B} + \overline{C})(\overline{A} + B + C)(\overline{A} + \overline{B} + C)$$

(2) 任意两个不同的最小项之积恒为0，任意两个不同的最大项之和恒为1，即如果 $i \neq j$ ，则有

$$m_i \cdot m_j = 0 \qquad M_i + M_j = 1$$

(3) 相同序号的最小项和最大项互为反函数，即

$$m_i = \overline{M_i}$$

$$m_i + M_i = 1$$

$$F(A, B, C) = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}C + ABC$$

$$F(A, B, C) = (A + B + \overline{C})(A + \overline{B} + \overline{C})(\overline{A} + B + C)(\overline{A} + \overline{B} + C)$$

#### 4) 标准积之和（与或）式与标准和之积（或与）式的关系

(1) 标准积之和式与标准和之积式是同一函数的两种不同表示形式，因此二者在**本质上是相等**的。

(2) 两种标准式中的最小项和最大项序号间存在一种**互补**关系，即**标准积之和式中未出现的最小项序号k必以最大项的序号k出现在标准和之积式中**，反之亦然。利用这一特性，可以方便地根据一种标准表达式写出另一种标准表达式。

(3) 由**相同自变量和相同序号**构成的最小项表达式与最大项表达式互为反函数。

例： 已知 $F(A,B,C) = \sum m(0,1,4,7)$ ， 写出其最大项表达式。

**解** 三变量函数的最小项或最大项序号可有0、1、2、3、4、5、6、7， 现在给定的最小项表达式中出现了序号0、1、4、7， 因此， 未出现的2、3、5、6等序号必出现在最大项表达式中。所以， 最大项表达式为

$$F(A,B,C) = \prod M(2,3,5,6)$$

下面证明其正确性。

因为  $F + \bar{F} = 1$

所以 
$$\begin{aligned}\sum m(0,1,4,7) + \bar{F} &= 1 = \sum m(0,1,2,3,4,5,6,7) \\ &= \sum m(0,1,4,7) + \sum m(2,3,5,6)\end{aligned}$$

因此  $\bar{F} = \sum m(2,3,5,6)$

$$F = \overline{\bar{F}} = \overline{\sum m(2,3,5,6)} = \prod \bar{m}(2,3,5,6) = \prod M(2,3,5,6)$$

# 真值表

- 用来反映变量所有取值组合及对应函数值的表格，称为真值表。

例：某公司有A、B、C三个股东，分别占有公司50%、30%和20%的股份。一个议案要获得通过，必须有超过50%股权的股东投赞成票。试列出该公司表决电路的真值表。

解 用1表示股东赞成议案，用0表示股东不赞成议案；用F表示表决结果，且用1表示议案获得通过，用0表示议案未获得通过。根据这些假定，不难列出该公司表决电路的真值表，如下表所示。



真值表

| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |



深圳大学  
SHENZHEN UNIVERSITY



# 数字电路

江常坤

# 卡诺图

- 卡诺图(Karnaugh Map)实际上是由真值表变换而来的一种方格图。卡诺图上的每一个小方格代表真值表上的一行，因而也就代表一个最小项或最大项。真值表有多少行，卡诺图就有多少个小方格。卡诺图不仅是逻辑函数的描述工具，而且还是逻辑函数化简的重要工具。

## 1) 二变量卡诺图

二变量卡诺图的结构如图所示。每个小方格中左上角的数字表明该小方格所表示的真值表中的行号，行号实际上就是真值表中自变量取值的等值十进制数，因而也就是它所代表的最小项或最大项的序号。

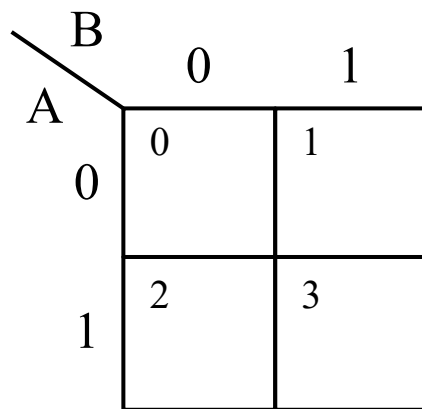
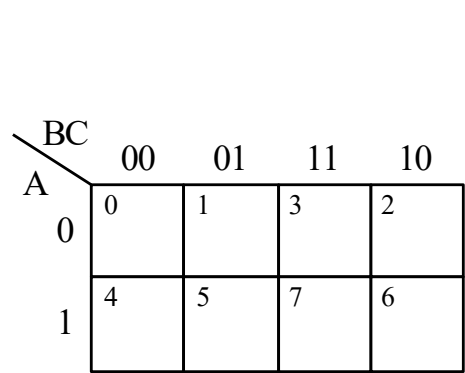
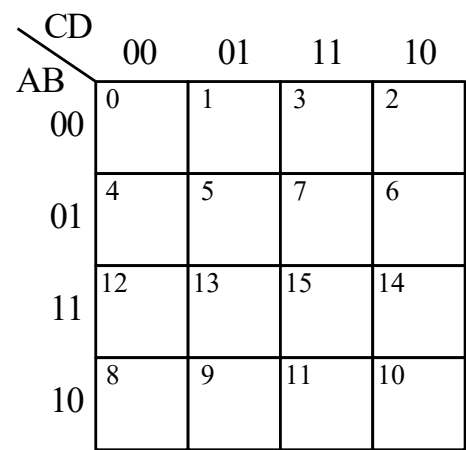


图 二变量卡诺图

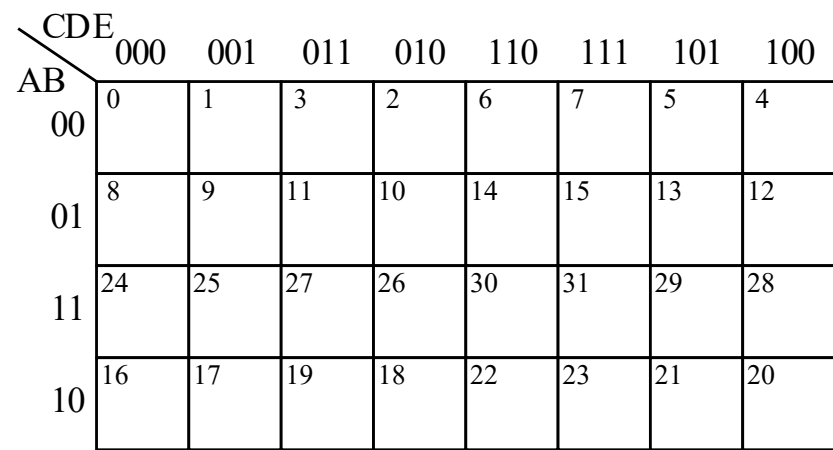
2) 多变量卡诺图



(a)



(b)



(c)

图 多变量卡诺图  
(a) 三变量； (b) 四变量； (c) 五变量

# 逻辑图

- 由逻辑门电路符号构成的，用来表示逻辑变量间关系的图形称为**逻辑电路图**。

图 1 - 7 所示为函数  $F = \overline{A}\overline{B} + \overline{A}(B+C)(C \oplus D)$  的逻辑图。

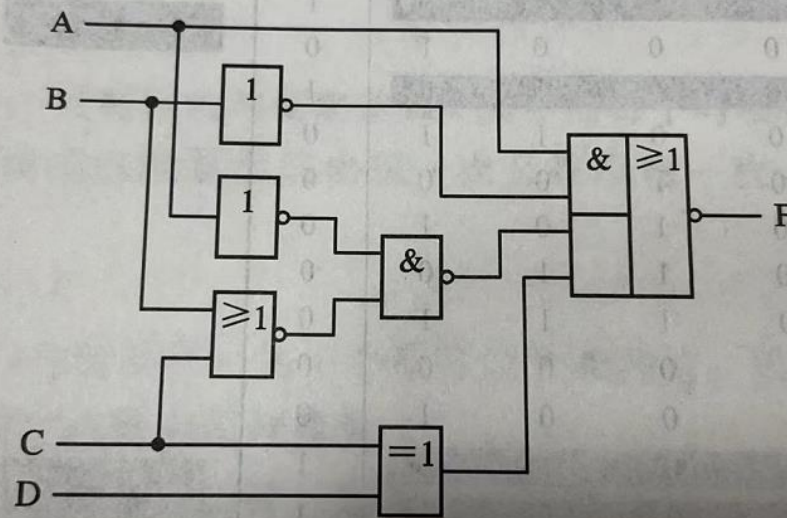
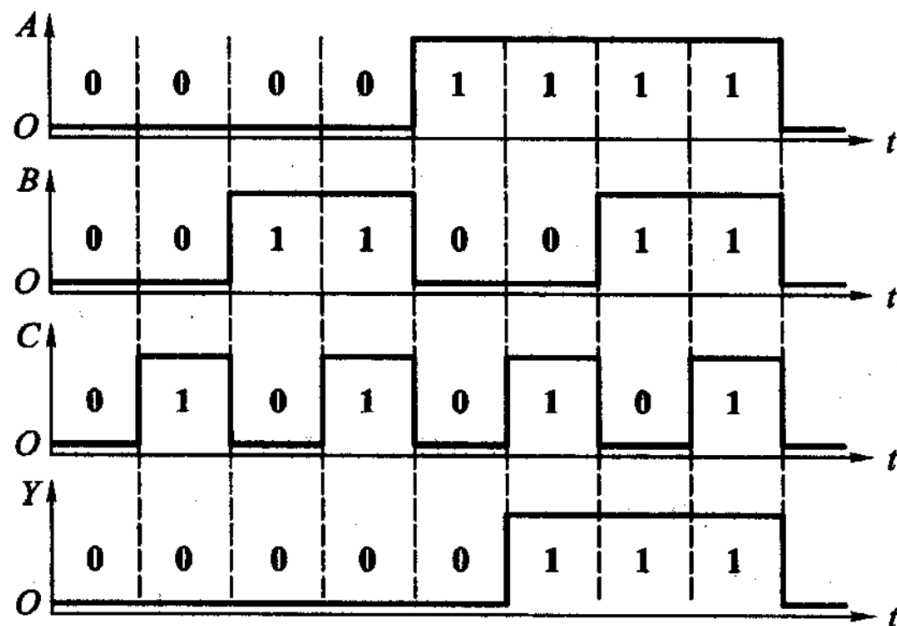


图 1 - 7 函数  $F = \overline{A}\overline{B} + \overline{A}(B+C)(C \oplus D)$  的逻辑图

# 波形图

- 将逻辑函数输入变量每一种可能出现的取值与对应的输出按时间顺序依次排列起来，称为逻辑函数的**波形图（时序图）**。

| 输 入 |   |   | 输 出 |
|-----|---|---|-----|
| A   | B | C | Y   |
| 0   | 0 | 0 | 0   |
| 0   | 0 | 1 | 0   |
| 0   | 1 | 0 | 0   |
| 0   | 1 | 1 | 0   |
| 1   | 0 | 0 | 0   |
| 1   | 0 | 1 | 1   |
| 1   | 1 | 0 | 1   |
| 1   | 1 | 1 | 1   |



# 不同描述方法间的转换

## 一。表达式→真值表

- 按照二进制码的顺序列出函数所含逻辑变量的所有不同取值组合，在确定出相应的函数值。



## 二。从真值表写逻辑函数的标准式

### 1) 从真值表写标准积之和（与或）式

标准积之和式中的最小项与真值表中 $F=1$ 的各行变量取值一一对应，因此，逻辑函数的标准积之和式就是真值表中使函数值为1的各个最小项之和。由此得出从真值表写标准积之和式的方法如下：

(1) 找出 $F = 1$ 的行；

(2) 对每个 $F = 1$ 的行，取值为1的变量用原变量表示，取值为0的变量用反变量表示，然后取其乘积，得到最小项；

(3) 将各个最小项进行逻辑加，得到标准积之和式。

## 2) 从真值表写标准和之积（或与）式

标准和之积式中的最大项与真值表中 $F=0$ 的各行变量取值一一对应，因此，**逻辑函数的标准和之积式就是真值表中使函数值为0的各个最大项之积**。由此得出从真值表写标准和之积式的方法如下：

(1) 找出 $F=0$ 的行；

(2) 对每个 $F=0$ 的行，取值为0的变量用**原**变量表示，取值为1的变量用**反**变量表示，然后取**其和**，得到最大项；

(3) 将各个最大项进行**逻辑乘**，得到标准和之积式。

逻辑函数的标准和之积式也可通过下面方法得到：首先写出 $\bar{F}$ 的标准积之和式，然后利用摩根定律求出  $F = \overline{\bar{F}}$  的标准和之积式。

### 3) 标准与或表达式和标准或与表达式之间的转换

同一函数，其标准与或表达式中最小项的编号和其标准或与表达式中最大项的编号是互补的。

例： 写出下表所示真值表的标准与或表达式和标准或与表达式。

真值表

| A | B | C | F | $\bar{F}$ |
|---|---|---|---|-----------|
| 0 | 0 | 0 | 0 | 1         |
| 0 | 0 | 1 | 0 | 1         |
| 0 | 1 | 0 | 0 | 1         |
| 0 | 1 | 1 | 0 | 1         |
| 1 | 0 | 0 | 0 | 1         |
| 1 | 0 | 1 | 1 | 0         |
| 1 | 1 | 0 | 1 | 0         |
| 1 | 1 | 1 | 1 | 0         |

解：

其标准与或表达式为

$$\begin{aligned} F(A, B, C) &= A\bar{B}C + AB\bar{C} + ABC = m_5 + m_6 + m_7 \\ &= \sum m(5, 6, 7) \end{aligned}$$

其标准或与表达式为

$$\begin{aligned} F(A, B, C) &= (A + B + C)(A + B + \bar{C})(A + \bar{B} + C) \\ &\quad (A + \bar{B} + \bar{C})(\bar{A} + B + C) \\ &= M_0 M_1 M_2 M_3 M_4 = \prod M(0, 1, 2, 3, 4) \end{aligned}$$

如果首先写出  $\overline{F}$  的标准积之和（与或）式，然后利用摩根定律，也可得到标准和之积（或与）式为

$$\overline{F}(A,B,C) = \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$

$$F(A,B,C) = \overline{\overline{F}} = \overline{\overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}}$$

$$= \overline{\overline{A}\overline{B}\overline{C}} \overline{\overline{A}\overline{B}C} \overline{\overline{A}B\overline{C}} \overline{\overline{A}BC} \overline{A\overline{B}\overline{C}}$$

$$= (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)(A + \overline{B} + \overline{C})(\overline{A} + B + C)$$

$$= \prod M(0,1,2,3,4)$$

当只需写出简记形式的标准积之和式和标准和之积式时，分别将  $F = 1$  和  $F = 0$  的各行中自变量取值化为十进制数，就可分别得到标准积之和式中各最小项的序号和标准和之积式中各最大项的序号。实际上，序号也是真值表从0开始编号的行号。

#### 4) 表达式与标准表达式之间的转换

与或表达式-〉标准与或表达式

与项中缺少的项  $A + \overline{A} = 1$

例1。 19

或与表达式-〉标准或与表达式

或项中缺少的项  $A \overline{A} = 0$

例1。 21

【例 1.19】 写出函数  $F = A + \bar{B}C + \bar{A}B\bar{C}$  的标准与或表达式。

解

$$F = A + \bar{B}C + \bar{A}B\bar{C}$$

$$= A(B + \bar{B})(C + \bar{C}) + (A + \bar{A})\bar{B}C + \bar{A}B\bar{C}$$

$$= ABC + AB\bar{C} + A\bar{B}C + A\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C}$$

$$= \bar{A}B\bar{C} + \bar{A}B\bar{C} + \bar{A}B\bar{C} + \bar{A}B\bar{C} + AB\bar{C} + ABC$$

也可以写成

$$F(A, B, C) = m_1 + m_2 + m_4 + m_5 + m_6 + m_7$$

或

$$F(A, B, C) = \sum m(1, 2, 4, 5, 6, 7)$$

或

$$F(A, B, C) = \sum (1, 2, 4, 5, 6, 7)$$

【例 1.21】 写出函数  $F = A(B + C)$  的标准或与表达式。

解

$$F = A(\bar{B} + C)$$

$$= (A + B\bar{B} + C\bar{C})(A\bar{A} + \bar{B} + C)$$

$$= (A + B + C)(A + \bar{B} + C)(A + B + \bar{C})(A + \bar{B} + \bar{C})(A + \bar{B} + C)(\bar{A} + \bar{B} + C)$$

$$= (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)(A + \bar{B} + \bar{C})(\bar{A} + \bar{B} + C)$$

也可以写成

$$F(A, B, C) = M_0 M_1 M_2 M_3 M_6$$

或

$$F(A, B, C) = \prod M(0, 1, 2, 3, 6)$$

或

$$F(A, B, C) = \prod (0, 1, 2, 3, 6)$$



### 三。从真值表到卡诺图

在真值表中找到函数值为1的变量组合，确定其大小标号，并在卡诺图中具有相应编号的方格中标上1，即可得到该函数的卡诺图。

例：用卡诺图描述下表所示真值表的逻辑函数。

**解** 表中所示真值表已编有行号，因此，将各行F取值填入三变量卡诺图编号相同的小方格即可，如图所示。有时候为了简洁起见，也可只填0或1。

| 行号 | A | B | C | F |
|----|---|---|---|---|
| 0  | 0 | 0 | 0 | 0 |
| 1  | 0 | 0 | 1 | 1 |
| 2  | 0 | 1 | 0 | 1 |
| 3  | 0 | 1 | 1 | 0 |
| 4  | 1 | 0 | 0 | 1 |
| 5  | 1 | 0 | 1 | 0 |
| 6  | 1 | 1 | 0 | 0 |
| 7  | 1 | 1 | 1 | 1 |

| BC<br>A |  | 00 | 01 | 11 | 10 |
|---------|--|----|----|----|----|
|         |  |    |    |    |    |
| 0       |  | 0  | 1  | 3  | 2  |
|         |  | 0  | 1  | 0  | 1  |
| 1       |  | 4  | 5  | 7  | 6  |
|         |  | 1  | 0  | 1  | 0  |

卡诺图

## 四。从卡诺图到真值表

从卡诺图中找出函数值为1的方格所对应的变量组合，并在真值表中让相应的组合的函数值为1。

图1-9

## 五。从逻辑表达式到卡诺图

标准函数表达式的卡诺图填写非常方便。如果是**最小项表达式**，则只要将最小项表达式中出现的序号对应的卡诺图编号小方格**填入1**即可；如果是**最大项表达式**，则只要将最大项表达式中出现的序号对应的卡诺图编号小方格**填入0**即可。

当逻辑函数表达式不是标准形式时，可逐项填写卡诺图，其方法是：对于“**与或型**”表达式，**每个与项中的原变量用1表示，反变量用0表示，在卡诺图上找到对应这些变量取值的小方格并填入1**；对于“**或与型**”表达式，**每个或项中的原变量用0表示，反变量用1表示，在卡诺图上找到对应这些变量取值的小方格并填入0**。全部的“与项”或“或项”填写完毕后，卡诺图即填写完毕。其他类型逻辑函数的卡诺图可以类似填写。

例： 用卡诺图分别描述下列逻辑函数。

$$Y(A,B,C) = \sum m(0,1,2,6)$$

$$Z(A,B,C) = \prod M(1,2,4,5)$$

解 函数Y和Z的卡诺图如图所示。

| BC |   | 00     | 01     | 11 | 10     |
|----|---|--------|--------|----|--------|
| A  | 0 | 0<br>1 | 1<br>1 | 3  | 2<br>1 |
|    | 1 | 4      | 5      | 7  | 6<br>1 |

(a)

| BC |   | 00     | 01     | 11 | 10     |
|----|---|--------|--------|----|--------|
| A  | 0 | 0      | 1<br>0 | 3  | 2<br>0 |
|    | 1 | 4<br>0 | 5<br>0 | 7  | 6      |

(b)

(a) Y; (b) Z

例： 分别将下列逻辑函数填入卡诺图。

$$Y(A, B, C) = \overline{A}\overline{B} + C$$

$$Z(A, B, C) = (\overline{A} + B)\overline{C}$$

解：

Y的卡诺图如图（a）所示。其中，Y的第一个与项A填出卡诺图第二行左侧的两个1，Y的第二个与项C填出卡诺图中间两列的四个1。

Z的卡诺图如图（b）所示。其中，Z的第一个或项+B填出卡诺图第二行左侧的两个0，Z的第二个或项填出卡诺图中间两列的四个0。

| BC |   | 00     | 01     | 11     | 10 |
|----|---|--------|--------|--------|----|
| A  | 0 | 0      | 1<br>1 | 3<br>1 | 2  |
|    | 1 | 4<br>1 | 5<br>1 | 7<br>1 | 6  |

(a)

| BC |   | 00     | 01     | 11     | 10 |
|----|---|--------|--------|--------|----|
| A  | 0 | 0      | 1<br>0 | 3<br>0 | 2  |
|    | 1 | 4<br>0 | 5<br>0 | 7<br>0 | 6  |

(b)

卡诺图  
(a) Y; (b) Z

## 六。从卡诺图到标准表达式

### 1。标准与或表达式

将卡诺图中函数值为1的方格对应的最小项相或。

### 2。标准或与表达式

将卡诺图中函数值为0的方格对应的最大项相与。



例：写出Y和Z的标准与或表达式和标准或与表达式。

| BC |   | 00     | 01     | 11     | 10 |
|----|---|--------|--------|--------|----|
| A  | 0 | 0      | 1<br>1 | 3<br>1 | 2  |
|    | 1 | 4<br>1 | 5<br>1 | 7<br>1 | 6  |

(a) Y

| BC |   | 00     | 01     | 11     | 10 |
|----|---|--------|--------|--------|----|
| A  | 0 | 0      | 1<br>0 | 3<br>0 | 2  |
|    | 1 | 4<br>0 | 5<br>0 | 7<br>0 | 6  |

(b) Z

Y和Z的标准表达式分别为

$$\begin{aligned}
 Y(A, B, C) &= \sum m(1, 3, 4, 5, 7) = \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}\overline{C} + A\overline{B}C + ABC \\
 &= \prod M(0, 2, 6) = (A + B + C)(A + \overline{B} + C)(\overline{A} + \overline{B} + C)
 \end{aligned}$$

$$\begin{aligned}
 Z(A, B, C) &= \sum m(0, 2, 6) = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + AB\overline{C} \\
 &= \prod M(1, 3, 4, 5, 7) \\
 &= (A + B + \overline{C})(A + \overline{B} + \overline{C})(\overline{A} + B + C)(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + \overline{C})
 \end{aligned}$$

作业:

P33: 1-9(5);  
1-10;

P34: 1-11(1)(7);  
1-12(2)(7);  
1-13.



深圳大学  
SHENZHEN UNIVERSITY



# 数字电路

江常坤

## 1.5 逻辑函数的化简

# 逻辑函数最简的标准

逻辑函数“最简”的标准与函数本身的类型有关。类型不同，“最简”的标准也有所不同。

“与或型”逻辑函数需要同时满足下列两个条件，方可称为“最简”：

- (1) 与项最少；
- (2) 每个与项中的变量数最少。

“或与型”逻辑函数需要同时满足下列两个条件，方可称为“最简”：

(1)或项最少；(2)每个或项中的变量数最少。

注意：同一类型的逻辑函数表达式有时候可能会有简单程度相同的多个最简式。这与化简时所使用的有关。

### 1.5.1 公式法化简逻辑函数

直接利用逻辑代数的基本公式，通过并项（如  $AB + A\bar{B} = A$  ）

、吸收(如  $A+AB=A$ )、消元(如  $A + \bar{A}B = A + B$  )、配项  
(如  $AB + \bar{A}C = AB + \bar{A}C + BC$  )等多种手段消去逻辑函数中多余的项或变量，以实现逻辑函数最简化的方法，称为公式化简法。

例。用代数法化简逻辑函数  $F = A\bar{B} + \bar{A}B + B\bar{C} + \bar{B}C$

解

$$F = A\bar{B}(C + \bar{C}) + \bar{A}B + (A + \bar{A})B\bar{C} + \bar{B}C \quad \text{配项, } A + \bar{A} = 1$$

$$= A\bar{B}C + A\bar{B}\bar{C} + \bar{A}B + AB\bar{C} + \bar{A}B\bar{C} + \bar{B}C \quad (\text{分配律})$$

$$= \bar{B}C(A + 1) + A\bar{C}(\bar{B} + B) + \bar{A}B(1 + \bar{C}) \quad (\text{提取公因式})$$

$$= \bar{B}C + A\bar{C} + \bar{A}B \quad (A + \bar{A} = 1, 1 + A = 1)$$



## 公式化简的一个技巧：

1) 教材介绍的各种公式法，本质上是逻辑函数最小项的组合方式的指定。如果找不到合适的公式，但题目还要求用公式法，那么可以用卡诺图辅助。

2) 先完成卡诺图化简，确定最小项的组合方式，然后一方面可以为公式选择提供思路，另一方面也可以直接把原函数写成最小项之和形式，然后根据指定的组合方式进行组合（利用并项法）。

## 1.5.2 逻辑函数的卡诺图化简法

### 1. 卡诺图化简逻辑函数的原理

根据逻辑代数的吸收定律：

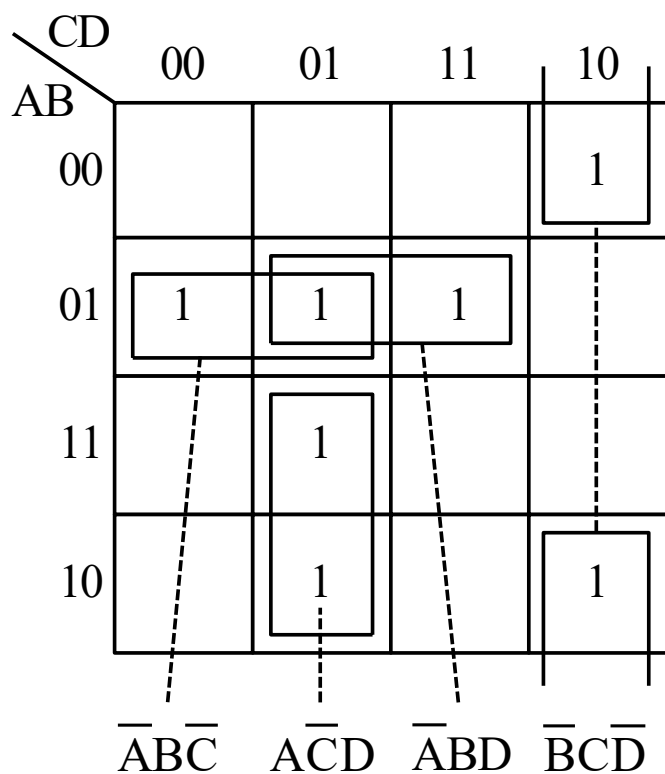
$$AB + A\bar{B} = A$$

$$(A + B)(A + \bar{B}) = A$$

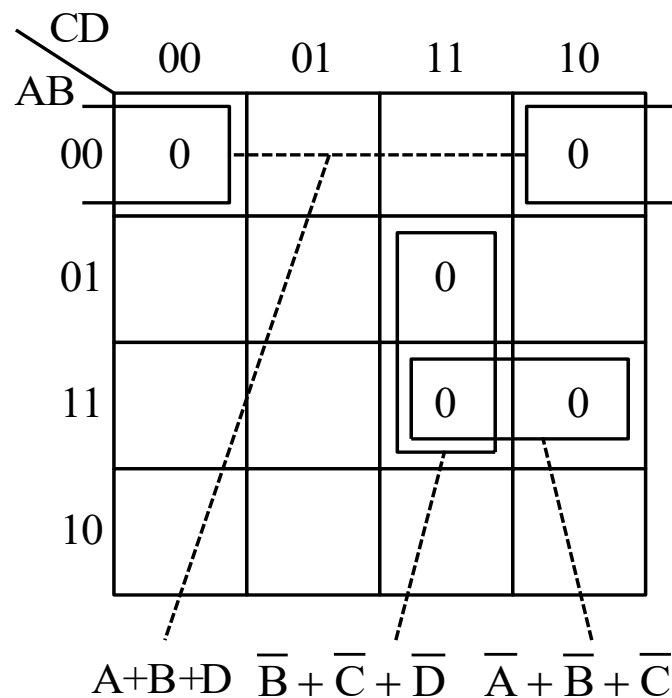
可知，任意两个只有一个变量取值不同的最小项或最大项结合，都可以消去取值不同的那个变量而合并为一项。而卡诺图上任意两个在几何位置上相邻或与中心轴对称的小方格代表的最小项或最大项都只有一个变量取值不同，因此，它们可以结合在一起，消除取值不同的那个变量而合并为一项。见图1-13。

## 2. 卡诺图上合并最小项(最大项)的规律

(1) 2个相邻的最小项(最大项)结合，可以消去1个取值不同的变量而合并为1项，如图所示：

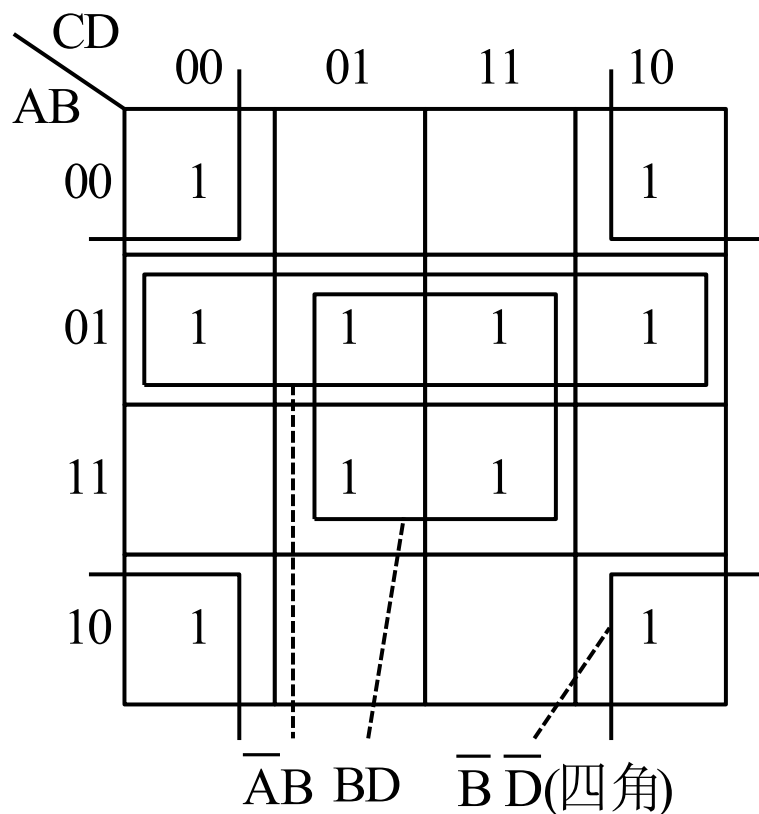


两个最小项结合

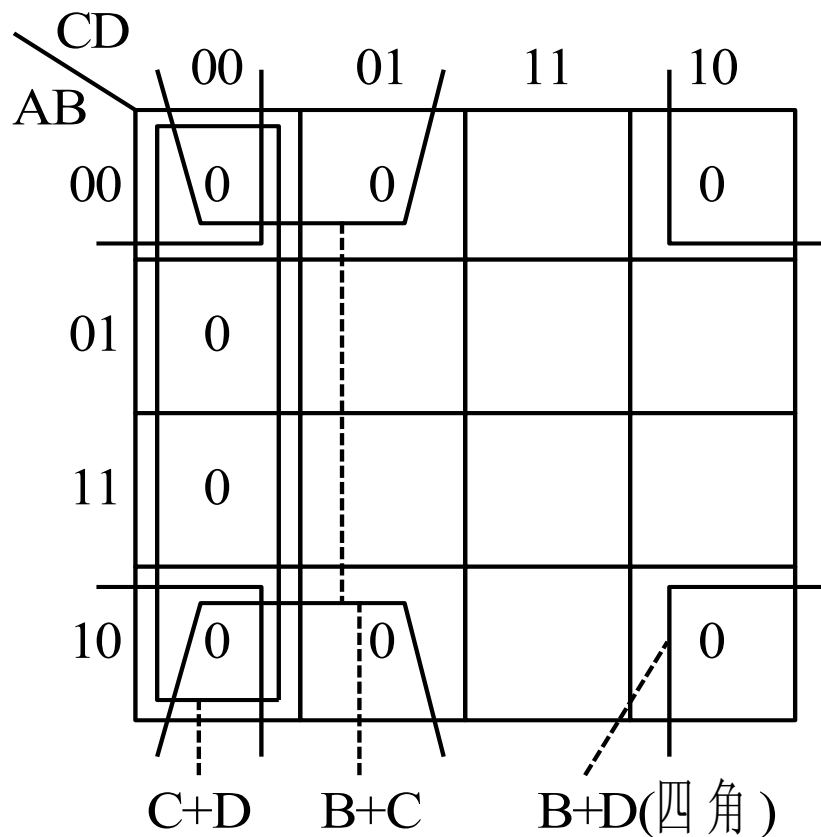


两个最大项结合

(2) 4个相邻的最小项(最大项)结合，可以消去2个取值不同的变量而合并为1项，如图所示；

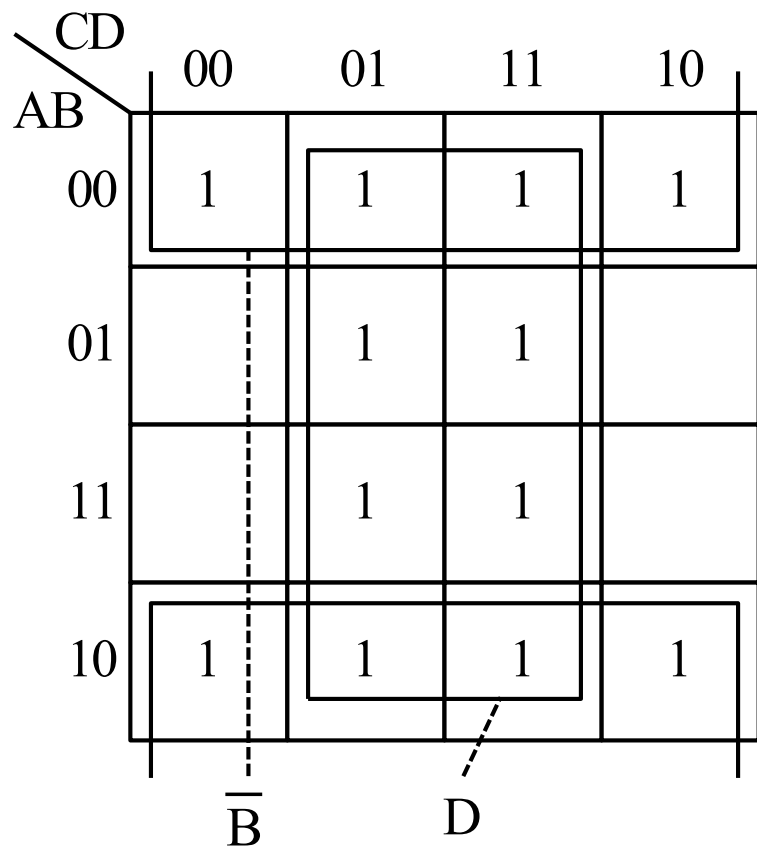


4个最小项结合

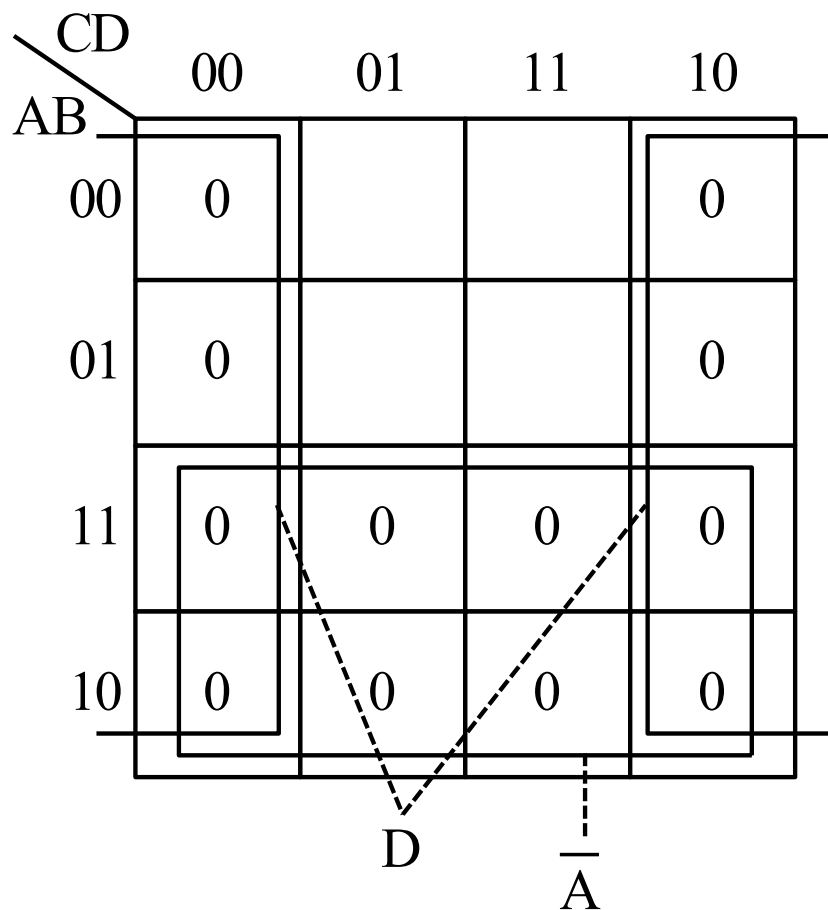


4个最大项结合

(3) 8个相邻的最小项(最大项)结合，可以消去3个取值不同的变量而合并为1项，如图所示；



8个最小项结合



8个最大项结合

(4) 一般结论:  $2^n$ 个相邻的最小项(最大项)结合, 可以消去  $n$ 个取值不同的变量而合并为1项。一个最小项(最大项)可以根据需要多次使用(因为  $A = A + A$ ,  $A = AA$ )。

(5) 所有方格为1, 全部圈在一起, 结果为1。

### 3. 卡诺图上合并最小项(最大项)的原则

为了保证在卡诺图上将逻辑函数化简到最简，画卡诺圈时应该遵循以下原则：

- (1) 从只有一种圈法或最少圈法的项开始。
- (2) 圈要尽可能大，圈的个数要尽可能少。每个圈内必须是 $2^n$ 个相邻项，且至少有一个最小项(最大项)为本圈所独有。
- (3) 卡诺图上所有的最小项(最大项)均被圈过。
- (4) 包括最小项（最大项）的圈中不含0（1）。

## 4. 卡诺图上化简逻辑函数的步骤

### (1) 画图：

根据给定真值表或逻辑表达式中显示的变量个数，画出相应的卡诺图。

### (2) 填图：

根据给定真值表或逻辑表达式填写卡诺图。为简便起见，可以只填1或0。

### (3) 画圈：

根据前述化简原则，画出所有的卡诺圈。求“最简与或式”时，圈1；求“最简或与式”时，圈0。



#### (4) 读图:

无论是圈0还是圈1，各个卡诺圈中取值不同的变量都将消去，只需读出取值相同的那些变量即可。

如果是圈1，取值为1的变量用原变量表示，取值为0的变量用反变量表示，取这些变量的乘积，即得该卡诺圈化简后的“最简积项”；将所有圈的“最简积项”逻辑加，即得“最简与或式”或“最简积之和式”。

如果是圈0，取值为0的变量用原变量表示，取值为1的变量用反变量表示，取这些变量的和，即得该卡诺圈化简后的“最简和项”；将所有圈的“最简和项”逻辑乘，即得“最简或与式”或“最简和之积式”。

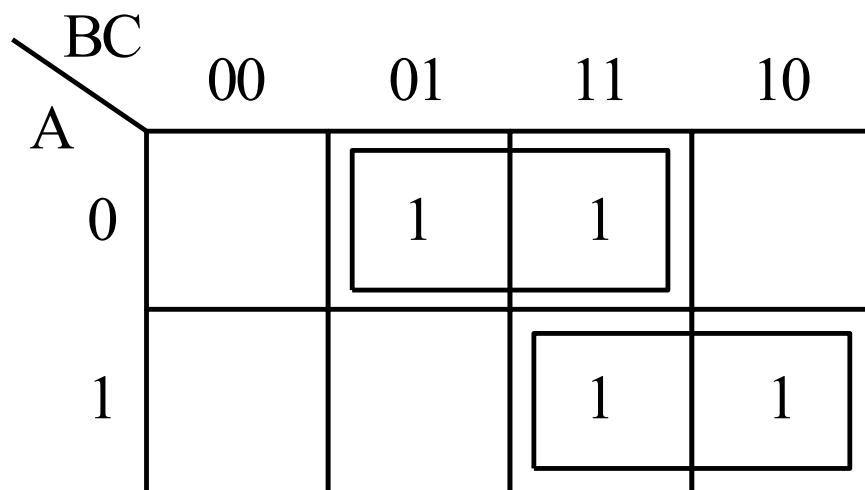
## 5. 化简举例

例： 用卡诺图化简逻辑函数  $F = AB + \overline{A}C + BC$  写出其最简与或式。

**解** 卡诺图如图所示。由图中可见，第三列的两个1均有两种圈法，而第二列和第四列的1都只有一种圈法，因此应从第二列或第四列的1开始圈。如果从第三列的两个1开始圈，很可能会增加第三列的一个圈。由此读出最简与或式为

$$F = AB + \overline{A}C$$

此化简结果与直接利用包含律得到的结果相同。



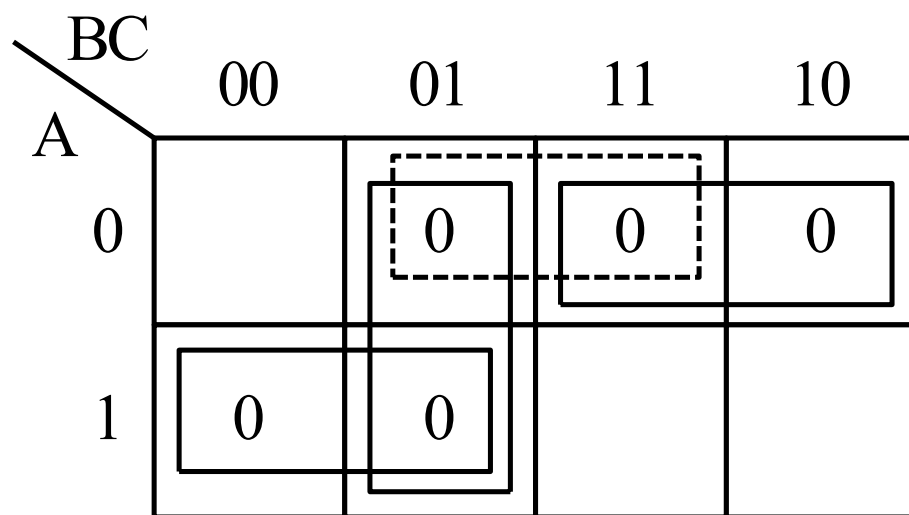
卡诺图

例：用卡诺图化简逻辑函数，  $F = (\bar{A} + B)(A + \bar{B})(A + B + \bar{C})$

写出其最简或与式。

**解** 卡诺图如图所示。由图中可见，除了第一列和第四列的0只有一种圈法外，其余的三个0均有两种圈法，因此应从第一列或第四列的0开始圈。这两个圈圈过后，剩下第一行左侧的0未圈。圈这个0的方法有两种，一种是与下方的0圈在一起（如实线框所示），另一种是与右侧的0圈在一起（如虚线框所示），这两种圈法的简单程度相同，可以任选一种圈法。选择前者时，最简或与式为

$$F = (\bar{A} + B)(A + \bar{B})(B + \bar{C})$$



卡诺图

例：分别用卡诺图化简下列逻辑函数并写出其最简与或式和或与式。

$$(1) Y(A,B,C,D) = \sum m(1,4,5,6,7,8,10,14)$$

$$(2) Z(A,B,C,D) = \prod M(0,1,2,6,8,10,11,12)$$

解 函数Y和Z的卡诺图如图所示。

Y的最简与或式（非惟一）和或与式分别为

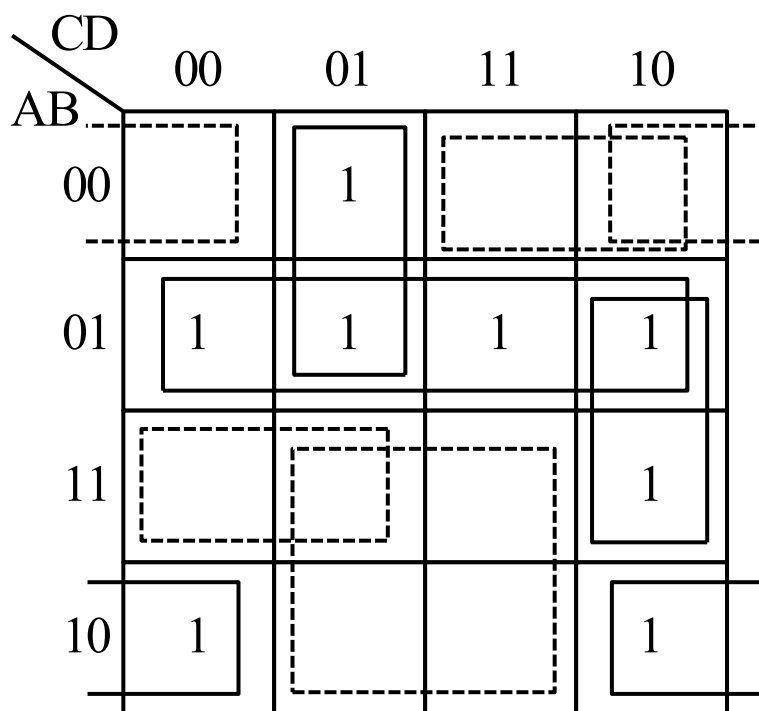
$$Y = \overline{A}\overline{C}D + \overline{A}B + BC\overline{D} + A\overline{B}\overline{D}$$

$$Y = (A + B + D)(A + B + \overline{C})(\overline{A} + \overline{B} + C)(\overline{A} + \overline{D})$$

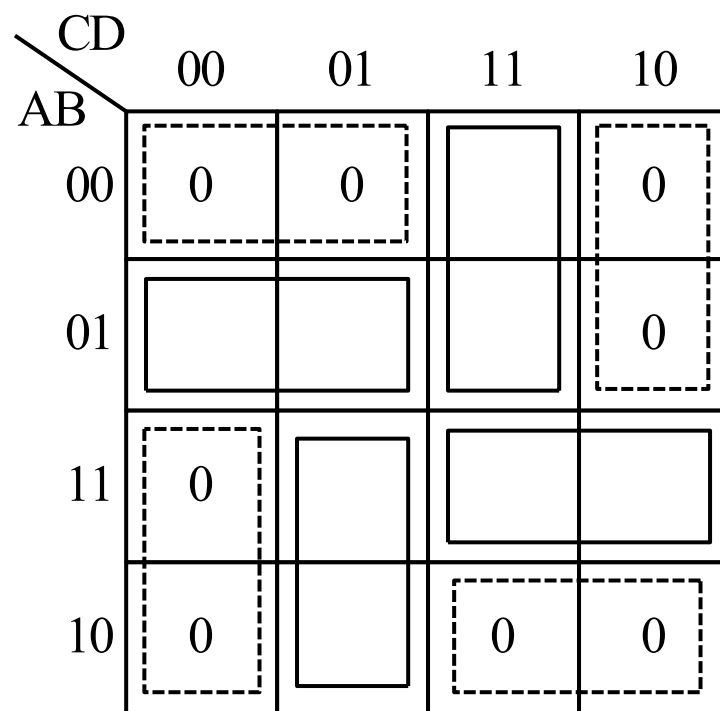
Z的最简与或式和或与式分别为

$$Z = \overline{A}CD + \overline{A}B\overline{C} + A\overline{C}D + ABC$$

$$Z = (A + B + C)(A + \overline{C} + D)(\overline{A} + C + D)(\overline{A} + B + \overline{C})$$



(a)



(b)

卡诺图

(a) Y;

(b) Z

例：用卡诺图化简下面的五变量逻辑函数并写出其最简或与式。

$$F(A,B,C,D,E) = \sum m(1,2,6,8,9,10,11,12,14,17,19,20,21,23,25,27,31)$$

解 F的卡诺图如图所示。需要注意的是，五变量逻辑函数化简时要特别小心。除了与中心轴对称的项相邻外，左右各四列构成的四变量卡诺图原有的相邻性仍然存在。如图所示卡诺图中，不仅第三、四行粗线框内的四个1相邻，而且第二行粗线框内的四个1也相邻。因此，F的最简积之和式为

$$F = \overline{C}\overline{D}\overline{E} + \overline{A}D\overline{E} + \overline{A}B\overline{C} + \overline{A}B\overline{E} + ADE + \overline{A}B\overline{C}\overline{D}$$



| CDE |    | 000    | 001     | 011     | 010     | 110     | 111     | 101     | 100     |
|-----|----|--------|---------|---------|---------|---------|---------|---------|---------|
| AB  | 00 | 0<br>1 | 1<br>1  | 3       | 2<br>1  | 6<br>1  | 7       | 5       | 4       |
|     | 01 | 8<br>1 | 9<br>1  | 11<br>1 | 10<br>1 | 14<br>1 | 15      | 13      | 12<br>1 |
|     | 11 | 24     | 25<br>1 | 27<br>1 | 26      | 30      | 31<br>1 | 29      | 28      |
|     | 10 | 16     | 17<br>1 | 19<br>1 | 18      | 22      | 23<br>1 | 21<br>1 | 20<br>1 |

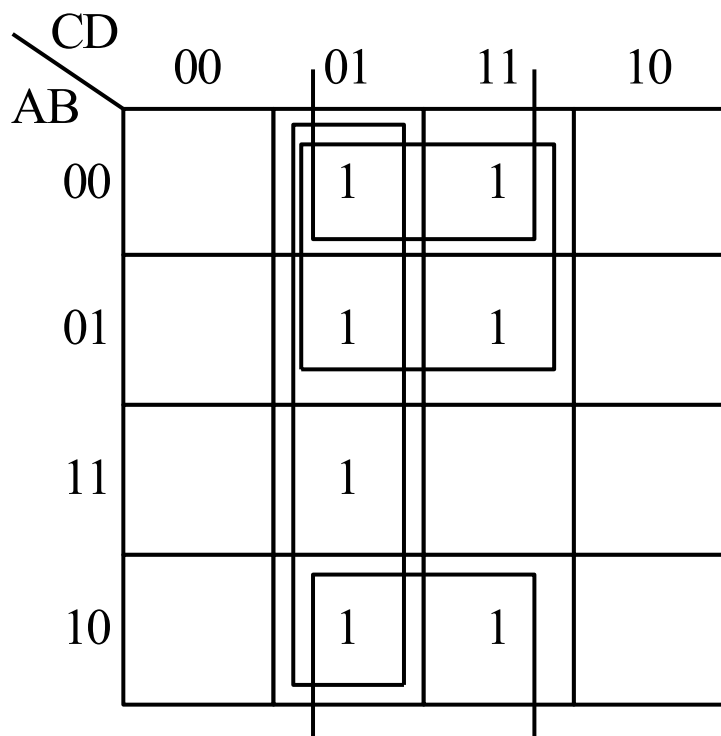
卡诺图

例：用卡诺图化简下面的多输出函数，要求总体最简。

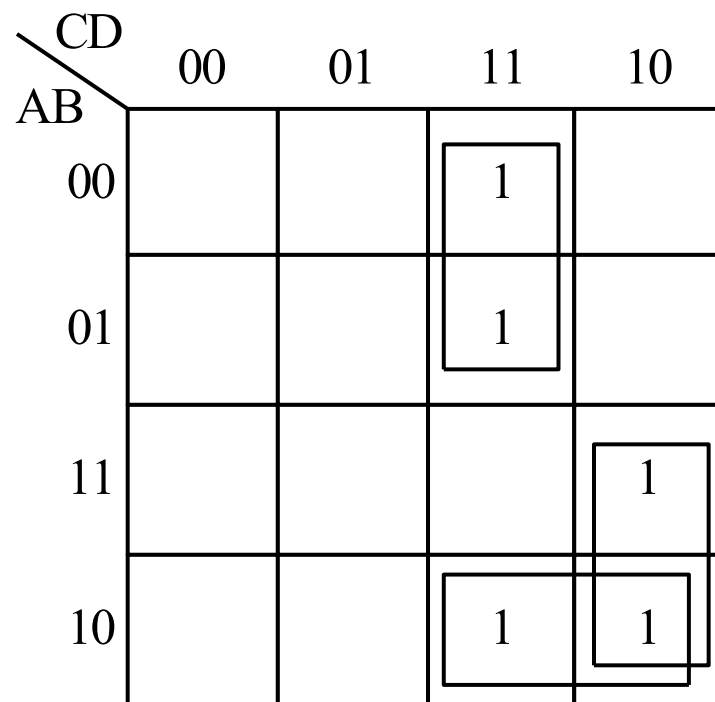
$$\begin{cases} F_1(A,B,C,D) = \sum m(1,3,5,7,9,11,13) \\ F_2(A,B,C,D) = \sum m(3,7,10,11,14) \end{cases}$$

**解** 当一个逻辑电路有多个输出函数时，需要追求的是整体最简而不是单个函数最简。化简时找出各个函数之间最恰当的公共项，有可能实现整体最简的目标。

当要求单个函数最简时， $F_1$ 、 $F_2$ 的卡诺图如图所示。



(a)



(b)

卡诺图(单个最简)

(a)  $F_1$ ; (b)  $F_2$

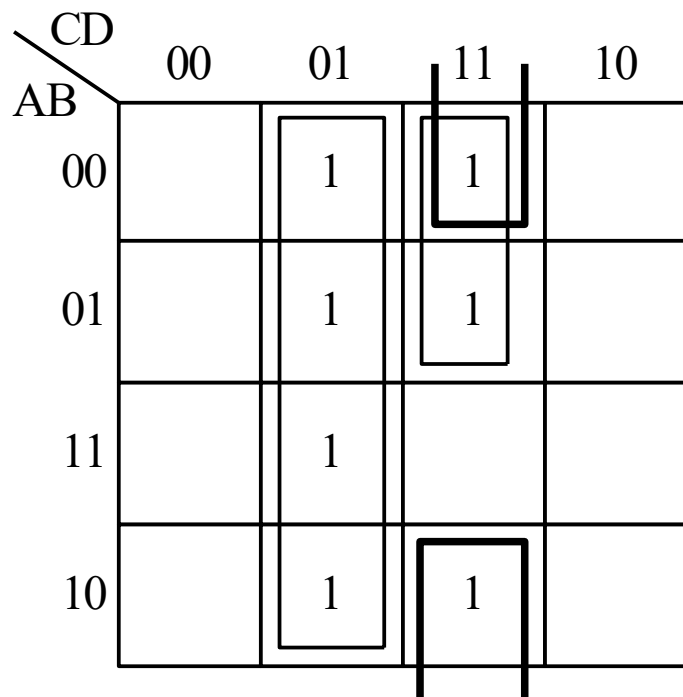
显然，此时的化简结果为

$$F_1 = \overline{A}D + \overline{B}D + \overline{C}D$$

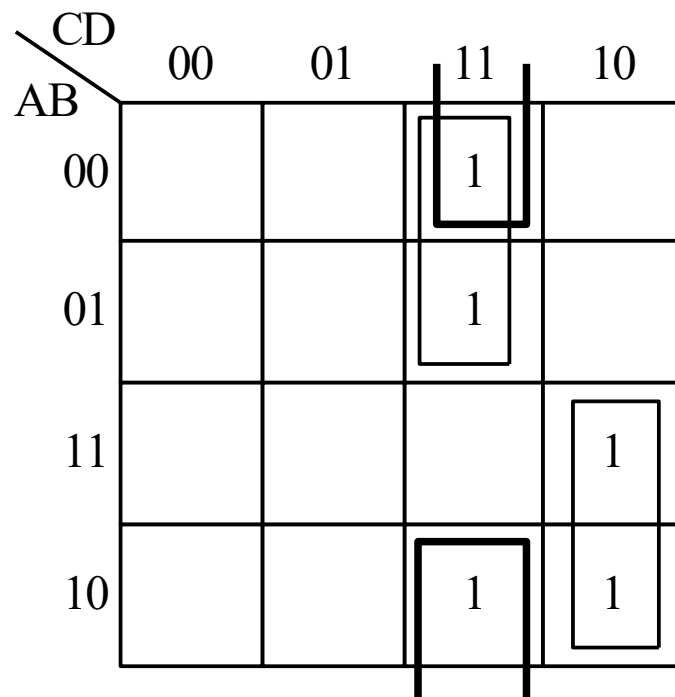
$$F_2 = \overline{A}CD + AC\overline{D} + A\overline{B}C$$

由表达式可见，实现函数功能需要3个2输入与门、3个3输入与门和两个3输入或门。如果将每一个与、或端均看作1个成本单位，则这种实现方案共需要8个逻辑门、21个成本单位。

当要求整体最简时，F1、F2的卡诺图如图所示(图中粗线框为公共圈)。



(a)



(b)

卡诺图(整体最简)

(a)  $F_1$ ;

(b)  $F_2$

此时的化简结果为

$$F_1 = \overline{A}CD + \overline{B}CD + \overline{C}D$$

$$F_2 = \overline{A}CD + \overline{B}CD + AC\overline{D}$$

由于有两个逻辑门共用，因此这种实现方案只需要3个3输入与门、1个2输入与门和两个3输入或门，即需要6个逻辑门、17个成本单位。可见该方案的整体成本要比单个最简的实现方案低一些。

另一种卡诺图化简为或与表达式的方法，例1.35

# 含有无关项的逻辑函数

## 无关项（Don't Care Terms）：约束项

例如,有三个逻辑变量  $A$ 、 $B$ 、 $C$ ,它们分别表示一台电动机的正转、反转和停止的命令, $A=1$ 表示正转, $B=1$ 表示反转, $C=1$ 表示停止。表示正转、反转和停止工作状态的逻辑函数可写成

$$Y_1 = A\bar{B}\bar{C}$$

正转

$$Y_2 = \bar{A}B\bar{C}$$

反转

$$Y_3 = \bar{A}\bar{B}C$$

停止

因为电动机任何时候只能执行其中的一个命令,所以不允许两个以上的变量同时为 1。 $ABC$  的取值只可能是 **001**、**010**、**100** 当中的某一种,而不能是 **000**、**011**、**101**、**110**、**111** 中的任何一种。因此, $A$ 、 $B$ 、 $C$  是一组具有约束的变量。

**结论：**根据最小项性质，每个最小项只有一组取值组合使它等于1，而对约束项来说，使它等于1的这组取值不可能出现。因此约束项恒等于0。对应真值表中，约束项对应的输出既可以是1也可以是0，一般用×或Φ表示。



# 含有无关项的逻辑函数

无关项（Don't Care Terms）：任意项

仍以上面的电动机正转、反转和停止控制为例。如果电路设计成当  $A$ 、 $B$ 、 $C$  三个控制变量出现两个以上同时为 1 或者全部为 0 时电路能自动切断供电电源,那么这时  $Y_1$ 、 $Y_2$  和  $Y_3$  等于 1 还是等于 0 已无关紧要,电动机肯定会受到保护而停止运行。例如,当出现  $A = B = C = 1$  时,对应的最小项  $ABC(m_7) = 1$ 。如果把最小项  $ABC$  写入  $Y_1$  式中,则当  $A = B = C = 1$  时  $Y_1 = 1$ ;如果没有把  $ABC$  这一项写入  $Y_1$  式中,则当  $A = B = C = 1$  时  $Y_1 = 0$ 。因为这时  $Y_1 = 1$  还是  $Y_1 = 0$  都是允许的,所以既可以把  $ABC$  这个最小项写入  $Y_1$  式中,也可以不写入。

结论：在某些变量取值下,任意项的值可以是1也可以是0。同时,函数值也是任意的（既可以是1也可以是0），真值表中同样用 $\times$ 或 $\Phi$ 表示。

无关项让函数值可以取1也可以取0，因此可以根据需要取1或0简化函数！

# 含有无关项的逻辑函数

## 无关项（Don't Care Terms）

|    | 约束项           | 任意项               |
|----|---------------|-------------------|
| 来源 | 输入组合物理上不可能出现  | 输入组合可能出现，但输出不影响功能 |
| 目的 | 避免无效输入导致的电路冗余 | 优化电路设计，降低复杂度      |

约束项是输入不可能出现时的“强制无关项”，需根据物理限制处理。

任意项是输出可自由定义的“灵活无关项”，需根据设计目标优化处理。

无关项代表的是逻辑上“永远不会出现”的状态：要么这些输入组合在现实中永远不会出现（约束）；要么是否出现不影响输出（任意）。

两者在化简时的核心思想相同：利用无关项的自由度，合并最小项，减少逻辑门数量。

# 含有无关项的逻辑函数

[例] 用卡诺图化简逻辑函数

$$\begin{cases} Y = F(A, B, C) = AC + \overline{A}\overline{B}C \\ \underline{\overline{B}C} = 0 \text{ 约束条件} \end{cases}$$

解：若不考虑约束项：

| A \ BC |    |    |    |    |
|--------|----|----|----|----|
|        | 00 | 01 | 11 | 10 |
| 0      | ×  | 1  |    |    |
| 1      | ×  | 1  | 1  |    |

$$Y = AC + \overline{B}C$$

考虑约束项：

| A \ BC |    |    |    |    |
|--------|----|----|----|----|
|        | 00 | 01 | 11 | 10 |
| 0      | ×  | 1  |    |    |
| 1      | ×  | 1  | 1  |    |

$$Y = AC + \overline{B}$$

更简

## 1.5.4 含有无关项的逻辑函数的化简

### 1. 含有无关项的逻辑函数的表示方法

含有无关项的逻辑函数通常有以下几种表示方法：

(1) 最小项表达式为

$$\begin{cases} F = \sum m(\quad) + \sum d(\quad) \\ \text{约束条件: } \sum d(\quad) = 0 \end{cases} \quad (1-11)$$

(2) 最大项表达式为

$$F = \prod M(\ ) \cdot \prod d(\ ) \quad (1-12)$$

或

$$\begin{cases} F = \prod M(\ ) \\ \text{约束条件: } \prod d(\ ) = 1 \end{cases} \quad (1-13)$$

### (3) 非标准表达式

此时，函数F和约束条件中至少有一个是非标准表达式，约束条件甚至可能是用语言来描述的。

上述三种表示方法中，括号内均为标准项的序号。无关项与最小项表达式是逻辑或的关系，而与最大项表达式是逻辑与的关系。将最小项表达式中的约束条件写成 $\sum d() = 0$ ，将最大项表达式中的约束条件写成 $\prod d() = 1$ ，相当于无关项的加入不会变原表达式所描述的逻辑电路的功能。注意约束条件的这两种不同表示方法。

## 2. 含有无关项的逻辑函数的化简

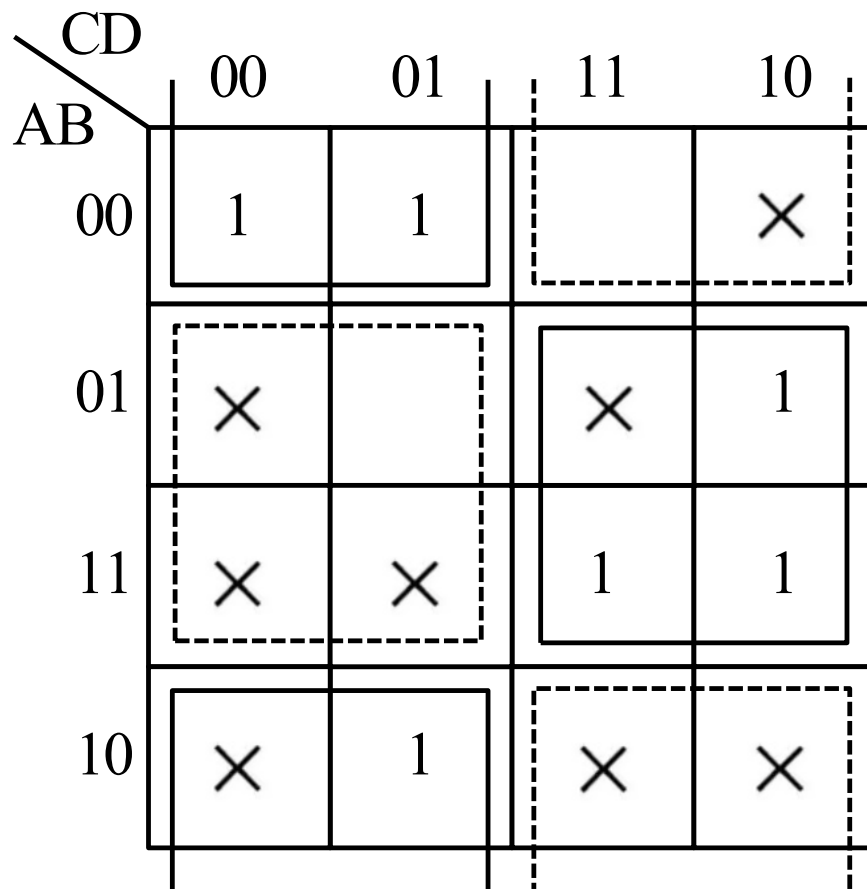
例：用卡诺图化简逻辑函数并写出其最简与或式和或与式。

$$F(A,B,C,D) = \sum m(0,1,6,9,14,15) + \sum d(2,4,7,8,10,11,12,13)$$

解 F的卡诺图如图所示。

最简与或式为  $F = \overline{B}\overline{C} + BC$

最简或与式为  $F = (\overline{B} + C)(B + \overline{C})$



卡诺图



**例：**用卡诺图下列化简逻辑函数，并写出其最简与或式和最简或与式。

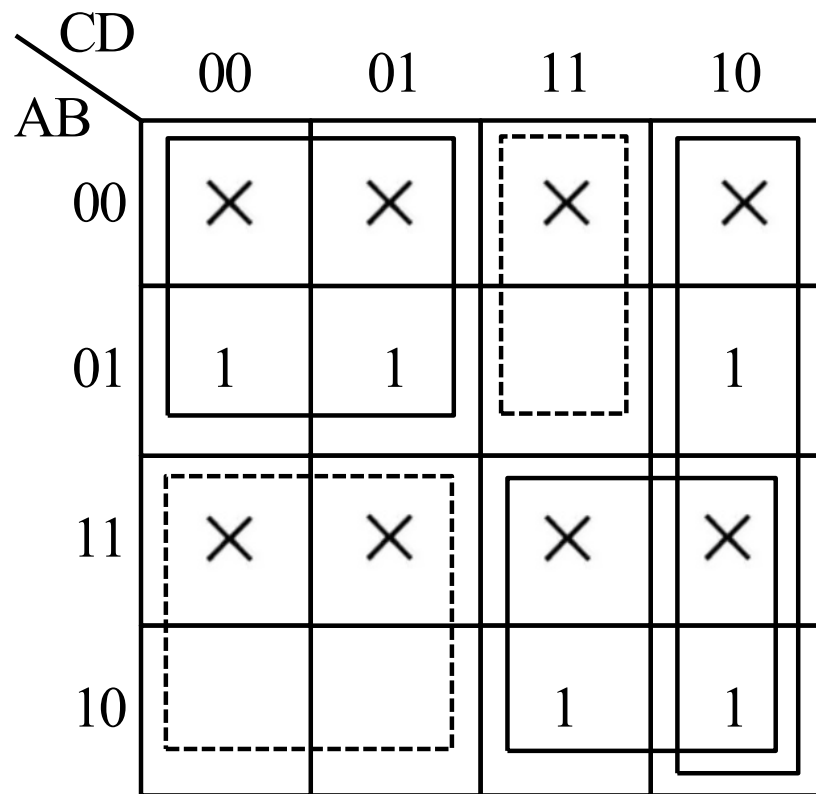
$$\begin{cases} F(A, B, C, D) = \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C}\overline{D} + A\overline{B}C\overline{D} + A\overline{B}CD \\ \text{约束条件: } A \odot B = 0 \end{cases}$$

**解** F的卡诺图如图所示，其最简与或式和最简或与式(均非惟一)分别为

$$F = \overline{A}\overline{C} + AC + C\overline{D}$$

$$F = (\overline{A} + C)(A + \overline{C} + \overline{D})$$

此题的约束条件也可用语言描述为：（物理条件）要求A、B不可能取值相同，即A、B取值相同的项为无关项。



卡诺图

例：某电路的输入ABCD是余3码，当余3码表示的十进制数是4或7的整数倍时，电路输出F为1， 否则F=0。 试列出该电路的真值表， 写出其最大项表达式， 并用卡诺图求出其最简与或式和最简或与式。

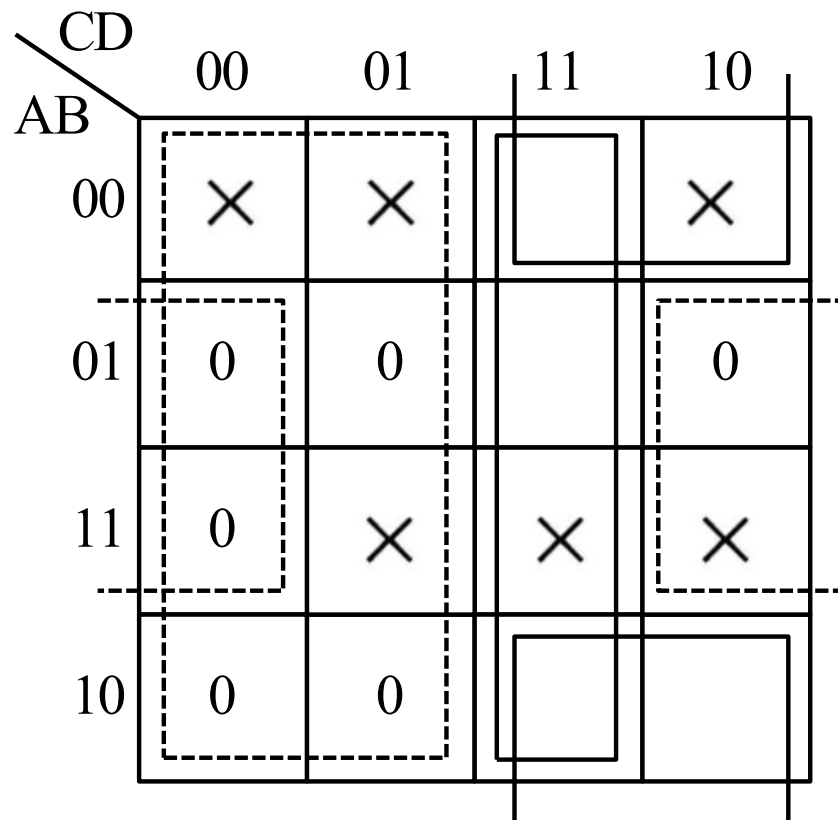
解 其真值表如表所示， 最大项表达式为

$$\begin{cases} F(A,B,C,D) = \prod M(4,5,6,8,9,12) \\ \text{约束条件: } \prod d(0,1,2,13,14,15)=1 \end{cases}$$

其卡诺图如图所示。 最简与或式和最简或与式分别为

$$F = \overline{B}C + CD$$

$$F = C(\overline{B} + D)$$



卡诺图

真值表

| $N_{10}$ | A | B | C | D | F |
|----------|---|---|---|---|---|
|          | 0 | 0 | 0 | 0 | × |
|          | 0 | 0 | 0 | 1 | × |
|          | 0 | 0 | 1 | 0 | × |
| 0        | 0 | 0 | 1 | 1 | 1 |
| 1        | 0 | 1 | 0 | 0 | 0 |
| 2        | 0 | 1 | 0 | 1 | 0 |
| 3        | 0 | 1 | 1 | 0 | 0 |
| 4        | 0 | 1 | 1 | 1 | 1 |
| 5        | 1 | 0 | 0 | 0 | 0 |
| 6        | 1 | 0 | 0 | 1 | 0 |
| 7        | 1 | 0 | 1 | 0 | 1 |
| 8        | 1 | 0 | 1 | 1 | 1 |
| 9        | 1 | 1 | 0 | 0 | 0 |
|          | 1 | 1 | 0 | 1 | × |
|          | 1 | 1 | 1 | 0 | × |
|          | 1 | 1 | 1 | 1 | × |

作业：

P34-36: 1-14(6)(8);  
1-15(5)(8);  
1-16(1)(6);  
1-17(2)(5);  
1-18(3).

# 第一章作业：

P32-36:

1-1(3); 1-2(3); 1-4(3);  
1-6(4); 1-7(7); 1-8(4);  
1-9(5); 1-10; 1-11(1)(7);  
1-12(2)(7); 1-13; 1-14(6)(8);  
1-15(5)(8); 1-16(1)(6);  
1-17(2)(5); 1-18(3).